



UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC

CIÊNCIA DA COMPUTAÇÃO

LAISA PEREIRA FRANÇA

LUCAS GABRIEL FERREIRA

ETAPA 1 — BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT

Escape-Time: Implementação sequencial e paralela em memória compartilhada com OpenMP.

ILHÉUS – BA

2026

LAISA PEREIRA FRANÇA
LUCAS GABRIEL FERREIRA

ETAPA 1 — BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT
Escape-Time: Implementação sequencial e paralela em memória compartilhada
com OpenMP

Estudo apresentado como requisito parcial à disciplina de Processamento Paralelo, DEC107, da Universidade Estadual de Santa Cruz, pelo curso de Ciência da Computação, turma de 2026.2.

Orientador (a): Prof. Esbel Tomas Valero Orellana.

ILHÉUS – BA

2026

SUMÁRIO

1. Introdução.....	5
2. Metodologia.....	7
3. Resultados.....	10
3.1 Teste de Corretude.....	10
3.2 Caso Base - Tempo de execução serial.....	10
3.3 Casos de Teste Schedule/Chunk.....	11
3.3.1 Representação Gráfica (Máquina 1).....	11
Speedup e Eficiência.....	12
Carga mínima, média e máxima.....	13
Fator de Balanceamento de Carga.....	13
3.3.2 Representação Gráfica (Máquina 2).....	14
Speedup e Eficiência.....	15
Carga mínima, média e máxima.....	16
Fator de Balanceamento de Carga.....	16
3.4 Casos de Estresse.....	17
3.4.1 Escalabilidade Forte (Casos de Teste).....	17
3.4.1.1 Representação Gráfica.....	18
Speedup e Eficiência.....	19
Carga mínima, média e máxima.....	20
Fator de Balanceamento de Carga.....	21
3.4.2 Desbalanceamento (Casos de Teste).....	21
3.4.2.1 Representação Gráfica.....	21
Speedup e Eficiência.....	23
Carga mínima, média e máxima.....	24
Speedup e Eficiência.....	26
Carga mínima, média e máxima.....	27
Fator de Balanceamento de Carga.....	27
3.4.3 Escalabilidade Fraca (Casos de Teste).....	28
3.4.3.1 Representação Gráfica.....	28
Speedup e Eficiência.....	30
Carga mínima, média e máxima.....	31
Fator de Balanceamento de Carga.....	32
3.4.5 Comparação entre máquinas (1 e 2).....	32
3.4.5.1 Representação Gráfica.....	32
Speedup e Eficiência.....	33
Carga mínima, média e máxima.....	34

Fator de Balanceamento de Carga.....	35
4. Discussão.....	36
4.1 Análise da Corretude da Implementação.....	36
4.2 Comparação entre a Versão Sequencial e a Versão Paralela em OpenMP.....	36
4.3 Análise do Impacto do Número de Threads no Desempenho.....	36
4.4 Discussão Fundamentada sobre o Balanceamento de Carga.....	37
4.5 Discussão de Gargalos Encontrados e Limitações da Implementação.....	38
4.6 Sugestões de Melhorias ou Otimizações Futuras.....	39
5. Conclusão.....	41
6. Referências.....	42
DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL.....	43
DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL.....	44

1. Introdução

O presente relatório tem como objetivo apresentar o desenvolvimento, o benchmark e a análise empírica da paralelização para a geração do Conjunto de Mandelbrot. O foco primário consiste na implementação de uma solução em C baseada em arquitetura de memória compartilhada, utilizando a interface de programação OpenMP. A meta é investigar como diferentes estratégias de distribuição de carga impactam a eficiência computacional de um problema embaraçosamente paralelo, estabelecendo métricas de desempenho robustas para a avaliação do sistema.

A fundamentação teórica para a geração do fractal apoia-se no algoritmo de tempo de escape (escape-time). Neste modelo, a imagem é tratada como um mapeamento do plano complexo, onde cada coordenada c (pixel) é submetida de forma independente à recorrência matemática $z_{n+1} = z_n^2 + c$, partindo de $z_0 = 0$. O processo iterativo de um pixel é encerrado assim que sua magnitude ultrapassa o raio de divergência, dada por $|z|^2 > 4$, ou quando um limite predefinido de iterações é atingido. Para explorar o paralelismo desse processo, o OpenMP é utilizado para particionar as iterações do laço espacial externo (linhas da matriz de imagem) entre múltiplas threads.

A eficiência da paralelização depende das políticas de escalonamento de laço (schedule) adotadas. O modelo estático (static) divide as iterações de forma determinística, agrupando-as em blocos (chunks) de tamanho fixo, o que minimiza o overhead do sistema, mas engessa a distribuição de trabalho. A política dinâmica (dynamic) adota uma abordagem sob demanda, na qual threads ociosas requisitam ativamente novos blocos de processamento em tempo de execução, incorrendo em um custo de gerenciamento maior em troca de flexibilidade. Por fim, a política guided aloca blocos dinamicamente com tamanhos que decrescem exponencialmente, buscando um balanço intermediário entre overhead e granulação.

Embora o cômputo dos pixels não apresente dependência de dados, o problema de Mandelbrot é caracterizado por um forte desbalanceamento em relação ao custo computacional por elemento. Pixels localizados nas extremidades do plano complexo divergem em poucas iterações. Em contrapartida, as coordenadas pertencentes ao interior do conjunto (como o cardioide principal) ou situadas em fronteiras específicas (como o "vale dos cavalos-marinheiros") mantêm as threads no laço iterativo até o limite máximo estabelecido. Essa disparidade espacial faz com que particionamentos estáticos e regulares da matriz tendam a gerar ociosidade sistêmica: threads responsáveis por regiões de escape rápido finalizam seus processamentos prematuramente e aguardam em barreiras de sincronização, enquanto o tempo total de execução é ditado por uma minoria de threads sobrecarregadas.

Por conseguinte, a análise e mitigação do desbalanceamento de carga configuram-se como o eixo central deste relatório. Em sistemas de memória compartilhada, avaliar o desempenho requer observar não apenas o tempo de execução global da aplicação, mas também o registro individual de trabalho processado por cada thread. A coleta

dessas métricas locais, correlacionada à topologia física do hardware utilizado, fornece os dados empíricos necessários para calcular e avaliar com precisão indicadores formais, como Speedup, Eficiência Paralela e o Fator de Balanceamento de Carga.

2. Metodologia

A condução do benchmark foi dividida em etapas de validação e medição de desempenho, apoiando-se no desenvolvimento de duas versões do algoritmo de geração do Conjunto de Mandelbrot na linguagem C, uma sequencial e uma paralela.

A implementação sequencial de referência foi estruturada com dois laços de repetição aninhados, iterando sobre as coordenadas y (linhas) e x (colunas) da imagem, mapeando cada pixel para sua respectiva coordenada no plano complexo e submetendo-o à função de tempo de escape. O resultado de cada coordenada foi armazenado em um vetor unidimensional adotando-se o mapeamento row-major (ordem de linha principal). Esta técnica é crucial para respeitar o princípio da localidade espacial, assegurando que elementos logicamente adjacentes na matriz sejam armazenados e acessados de forma contínua, maximizando o aproveitamento da memória cache. Para assegurar a corretude lógica e matemática da versão paralela, a saída desta implementação sequencial foi gravada em um arquivo binário bruto (.bin). A validação foi automatizada por meio da ferramenta de linha de comando cmp, verificando se os arquivos gerados pelas execuções seriais e paralelas eram bit a bit idênticos, atestando a ausência de condições de corrida (race conditions).

Na implementação paralela, a interface OpenMP foi utilizada para distribuir a carga computacional. A diretiva `#pragma omp parallel` foi empregada para a criação do grupo de threads, enquanto o fatiamento do trabalho foi direcionado ao laço externo (eixo y) utilizando a diretiva `#pragma omp for schedule(runtime)`. A escolha de paralelizar o laço de linhas manteve a aderência ao acesso row-major, preservando os ganhos de localidade de dados estabelecidos na versão sequencial. A adoção da cláusula `schedule(runtime)` foi uma decisão fundamental, pois transferiu a definição da política de escalonamento para o momento da execução (através da variável de ambiente `OMP_SCHEDULE`), permitindo a realização de testes em lote de forma flexível.

Para realizar a análise detalhada do balanceamento de carga sem introduzir contenção de memória, vetores de monitoramento indexados pelo identificador único de cada thread (tid) foram alocados. Essa abordagem permitiu que cada unidade de processamento registrasse em escopo local e independente a sua quantidade de pixels processados, iterações acumuladas e o tempo gasto no laço, evitando concorrência no acesso aos dados profiláticos que pudessem mascarar os tempos reais de execução.

As políticas de escalonamento avaliadas compreenderam os modelos static (com distribuição em blocos predefinidos), dynamic (distribuição de carga sob demanda) e guided (distribuição sob demanda com tamanho de bloco decrescente). Para analisar a influência da granularidade do particionamento no overhead do sistema, os tamanhos de fatia (chunks) testados foram: o valor padrão do OpenMP, 1, 8, 32 e 128 linhas.

Os testes foram conduzidos em duas máquinas distintas rodando o subsistema Windows Subsystem for Linux (WSL 2, Ubuntu 24.04), compilados com GCC (versão 13.3.0). A heterogeneidade do hardware permitiu avaliar o código em cenários

arquiteturais variados. Na máquina 1, o ambiente de execução possuía um Processador Intel Core i3-1115G4 (11ª Geração, arquitetura homogênea) com 2 núcleos físicos operando a 3GHz, totalizando 4 threads lógicas via Hyper-Threading, e 20 GB de memória RAM. Já na máquina 2, era um Processador Intel Core i5-12450H (12ª Geração, arquitetura híbrida) com 8 núcleos físicos operando a 2GHz, divididos em 4 núcleos de alta performance (P-Cores, com suporte a Hyper-Threading) e 4 núcleos de alta eficiência (E-Cores), totalizando 12 threads lógicas, e 20 GB de memória RAM.

Os procedimentos para a medição de tempo foram rigorosamente isolados das operações de entrada e saída (I/O) e da alocação inicial de memória. O cronômetro de precisão `omp_get_wtime()` foi posicionado imediatamente antes da diretiva de paralelização e congelado logo após a barreira de sincronização final, garantindo que exclusivamente o esforço computacional fosse contabilizado.

A avaliação de desempenho baseou-se nas seguintes métricas formais:

Tempo de Execução: Tempo total gasto pela região paralela para concluir o cálculo do fractal.

Speedup (S): Mede o ganho de velocidade, calculado pela razão entre o tempo sequencial de referência T_{seq} e o tempo da execução paralela com n threads T_{par} .

$$S = \frac{T_{seq}}{T_{par}}$$

Eficiência (E): Expressa a fração do tempo em que as unidades de processamento foram aproveitadas, obtida pela divisão do Speedup pelo número de threads utilizadas.

$$E = \frac{S}{n} \times 100\%$$

Balanceamento de Carga: Métrica central no problema do Mandelbrot para diagnosticar a eficácia da distribuição de trabalho. Para uma análise minuciosa, especialmente nos cenários de desbalanceamento acentuado, a métrica foi desdobrada em duas abordagens relativas. A razão entre o tempo máximo pelo tempo médio revela o distanciamento da thread gargalo (a mais lenta, que dita o tempo final) em relação ao cenário ideal de divisão perfeitamente equitativa. Valores próximos a 1.0 indicam um sistema otimizado e balanceado.

$$\text{Razão de Desbalanceamento} = \frac{T_{max}}{T_{media}}$$

A razão entre o tempo máximo e o tempo mínimo expõe a disparidade absoluta e a ociosidade interna do sistema. Mede diretamente a relação entre o esforço da thread mais sobrecarregada e da thread que finalizou o trabalho prematuramente. Valores

elevados denotam grave ineficiência no particionamento espacial imposto pelo algoritmo.

$$\textit{Disparidade de Carga} = \frac{T_{max}}{T_{min}}$$

3. Resultados

3.1 Teste de Corretude

A corretude da implementação paralela foi verificada por meio da comparação direta entre o arquivo binário contendo a matriz de contagens de iteração gerada pela versão sequencial e o arquivo correspondente produzido pela versão paralela em OpenMP. Para isso, foi utilizado o comando `cmp mandelbrot_matriz.bin mandelbrot_paralelo_matriz.bin && echo "IGUAIS" || echo "DIFERENTES"`. O utilitário `cmp` realiza uma comparação byte a byte entre os arquivos. O resultado obtido foi "IGUAIS", indicando que não houve divergência entre as matrizes geradas pelas duas implementações. Dessa forma, a paralelização preservou o resultado numérico da versão sequencial de referência.

```
laisaf@Laisa:/mnt/c/Users/Laisa França/Documents/Processamento Paralelo - 2026.2/Mandelbrot-PP$ cmp mandelbrot_matriz.bin mandelbrot_paralelo_matriz.bin && echo "IGUAIS" || echo "DIFERENTES"
IGUAIS
laisaf@Laisa:/mnt/c/Users/Laisa França/Documents/Processamento Paralelo - 2026.2/Mandelbrot-PP$
```

Figura 1 – Resultado do teste de corretude entre as versões sequencial e paralela.

3.2 Caso Base - Tempo de execução serial

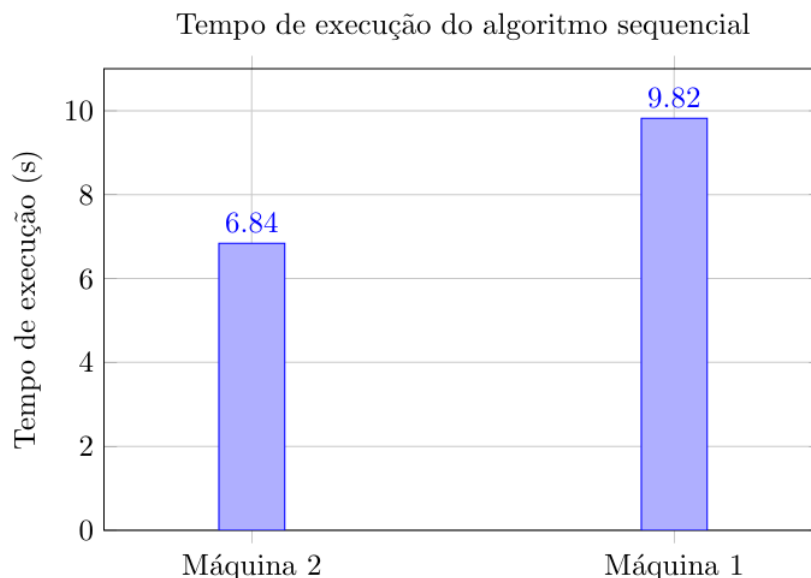


Figura 2 – Comparação do Tempo de Execução da versão sequencial do Mandelbrot para resolução 4096x4096 e $MAX_ITER = 1000$.

3.3 Casos de Teste Schedule/Chunk

Tabela 1: Resultados da avaliação das políticas de escalonamento e tamanhos de chunk na Máquina 1, utilizando resolução 4096x4096, MAX_ITER = 1000 e 4 threads.

Schedule	Chunk	Tempo (s)	Speedup	Eficiência (%)	Fator de Balanceamento
static	padrão	5.122935	1.917	47.92	1.941054
static	1	2.895508	3.391	84.78	1.004272
static	8	2.986903	3.287	82.19	1.014827
static	32	3.007078	3.265	81.63	1.012863
static	128	2.805620	3.500	87.50	1.022525
dynamic	1	2.804848	3.501	87.52	1.000379
dynamic	8	2.809629	3.495	87.37	1.000025
dynamic	32	2.816590	3.486	87.16	1.000518
dynamic	128	2.805852	3.500	87.49	1.020818
guided	1	4.475943	2.194	54.84	1.657805
guided	8	4.517961	2.173	54.33	1.667524
guided	32	4.765043	2.061	51.52	1.640313
guided	128	4.471820	2.196	54.89	1.663811

3.3.1 Representação Gráfica (Máquina 1)

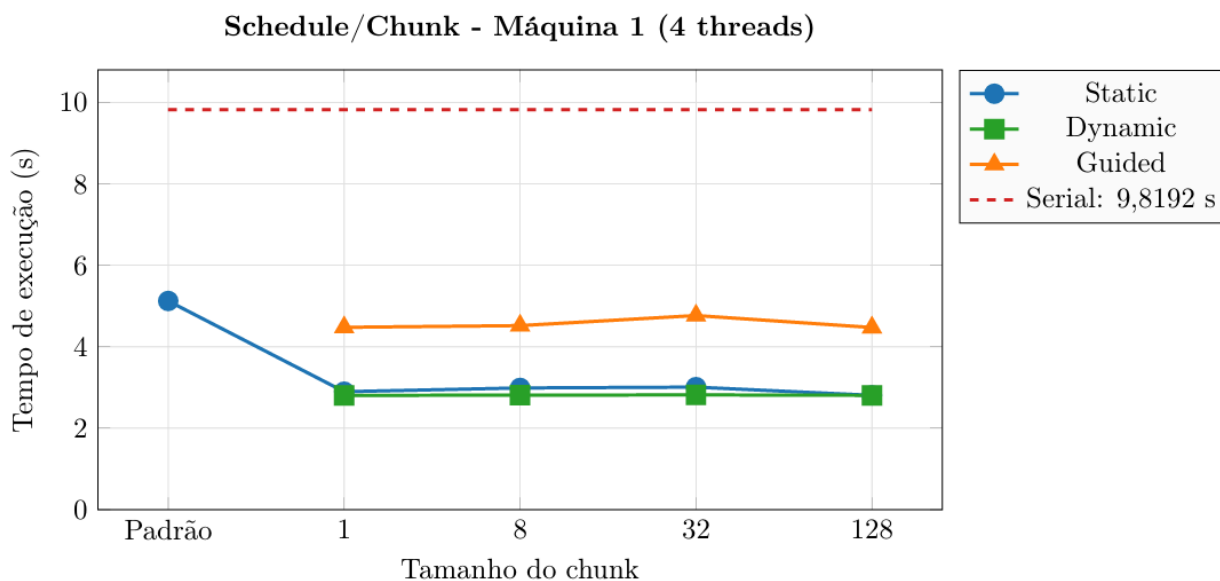


Figura 3 – Comparação do Tempo de Execução das políticas de escalonamento na Máquina 1. A linha vermelha tracejada representa o tempo da versão sequencial de referência.

Speedup e Eficiência

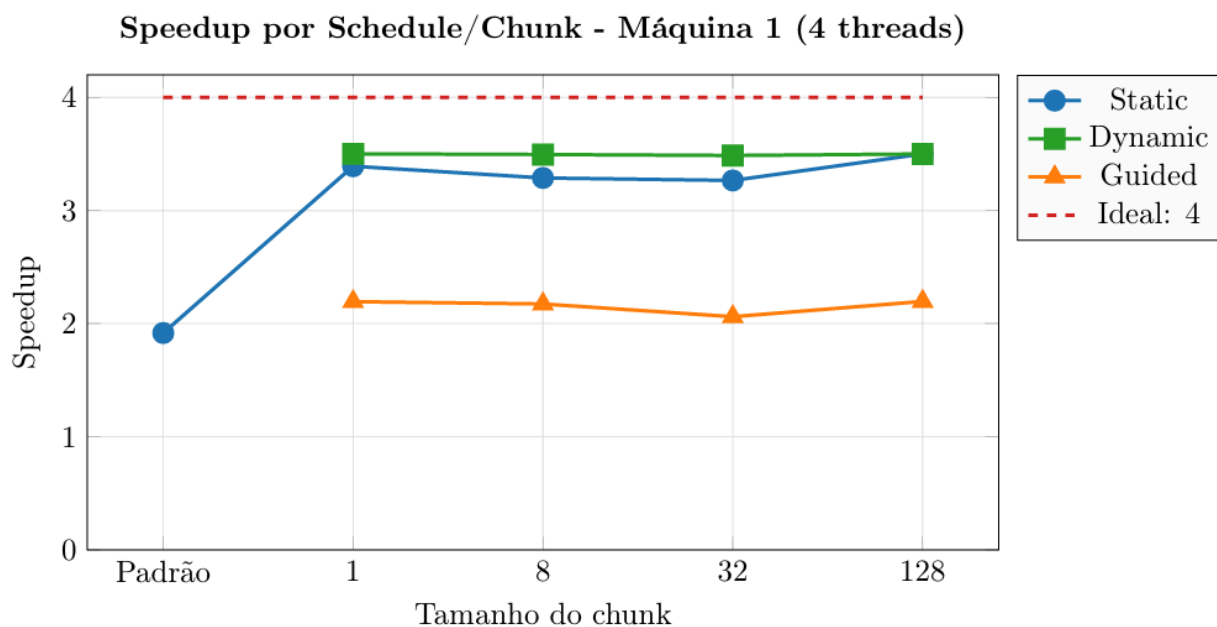


Figura 4 – Speedup obtido para as diferentes políticas de escalonamento na Máquina 1. O Speedup foi calculado com base no tempo sequencial de referência 9.82s.

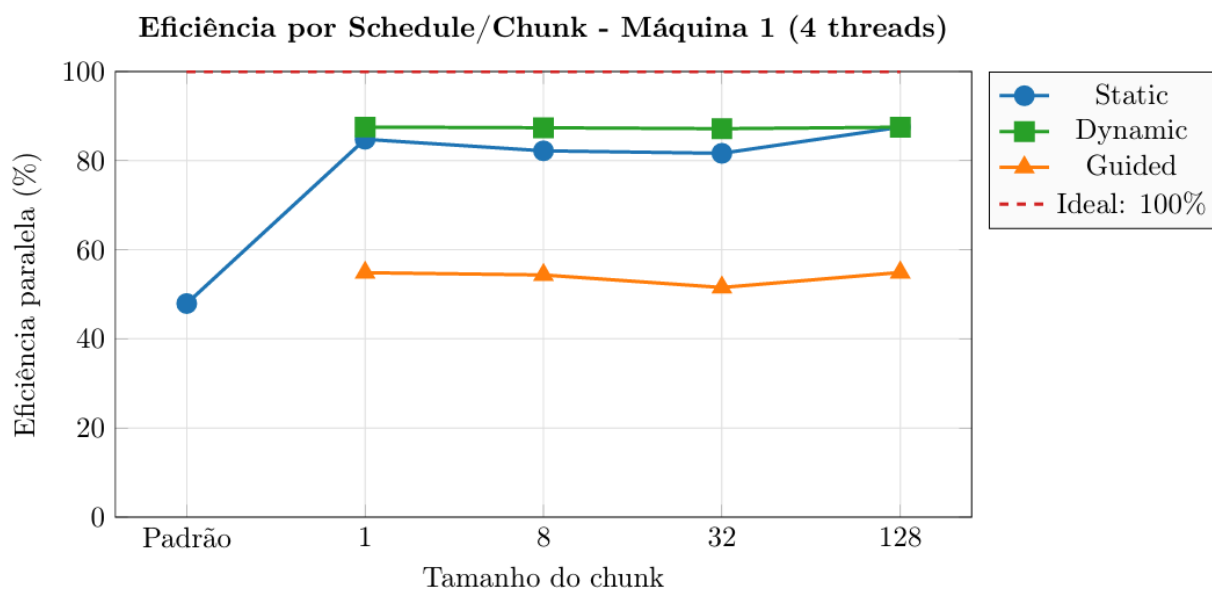


Figura 5 – Eficiência Paralela obtida para as diferentes políticas de escalonamento na Máquina 1.

Carga mínima, média e máxima

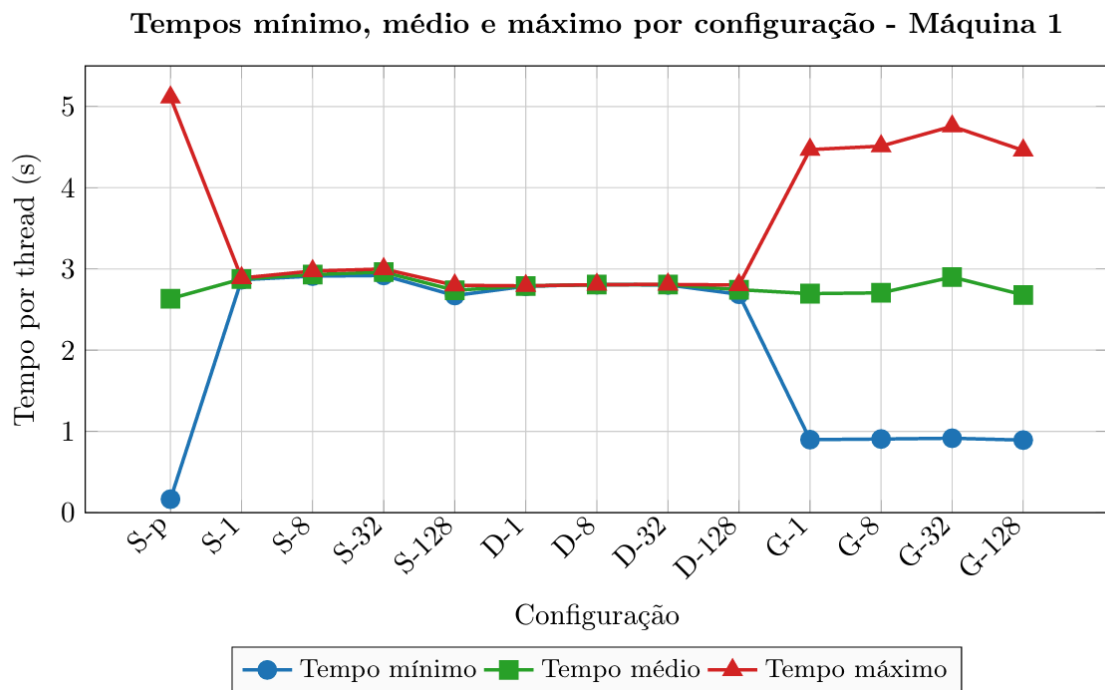


Figura 6 – Cargas obtidas para as diferentes políticas de escalonamento na Máquina 1.

Fator de Balanceamento de Carga

Fator de Balanceamento por Schedule/Chunk - Máquina 1 (4 threads)

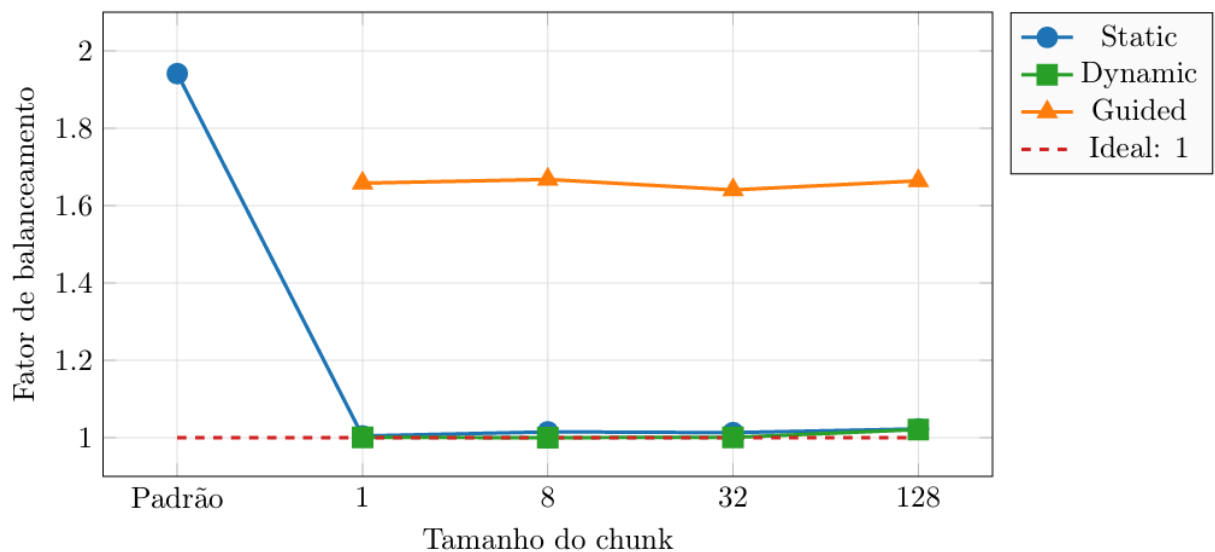


Figura 7 – Fator de Balanceamento de Carga obtido para as diferentes políticas de escalonamento na Máquina 1.

Tabela 2: Resultados da avaliação das políticas de escalonamento e tamanhos de chunk na Máquina 2, utilizando resolução 4096x4096, $MAX_ITER = 1000$ e 12 threads.

Schedule	Chunk	Tempo (s)	Speedup	Eficiência (%)	Fator de Balanceamento
static	padrão	1.827644	3.742	31.18	2.935236
static	1	0.818385	8.356	69.63	1.133878
static	8	0.806982	8.474	70.62	1.118973
static	32	0.812688	8.414	70.12	1.109293
static	128	0.905091	7.555	62.96	1.274240
dynamic	1	0.716367	9.546	79.55	1.000043
dynamic	8	0.747166	9.152	76.27	1.001721
dynamic	32	0.726721	9.410	78.42	1.002393
dynamic	128	0.873074	7.832	65.27	1.218830
guided	1	1.275086	5.363	44.69	1.702627
guided	8	1.134441	6.028	50.23	1.662335
guided	32	1.118448	6.114	50.95	1.617149
guided	128	1.217209	5.618	46.82	1.683403

3.3.2 Representação Gráfica (Máquina 2)

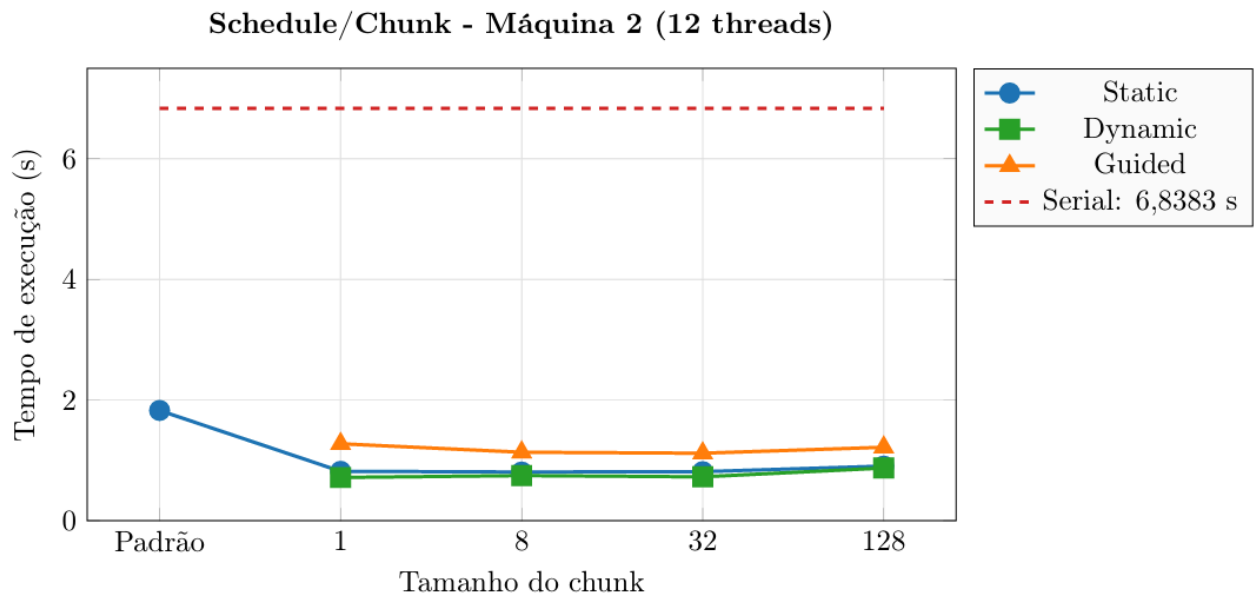


Figura 8 – Comparação do Tempo de Execução das políticas de escalonamento na Máquina 2. A linha vermelha tracejada representa o tempo da versão sequencial de referência.

Speedup e Eficiência

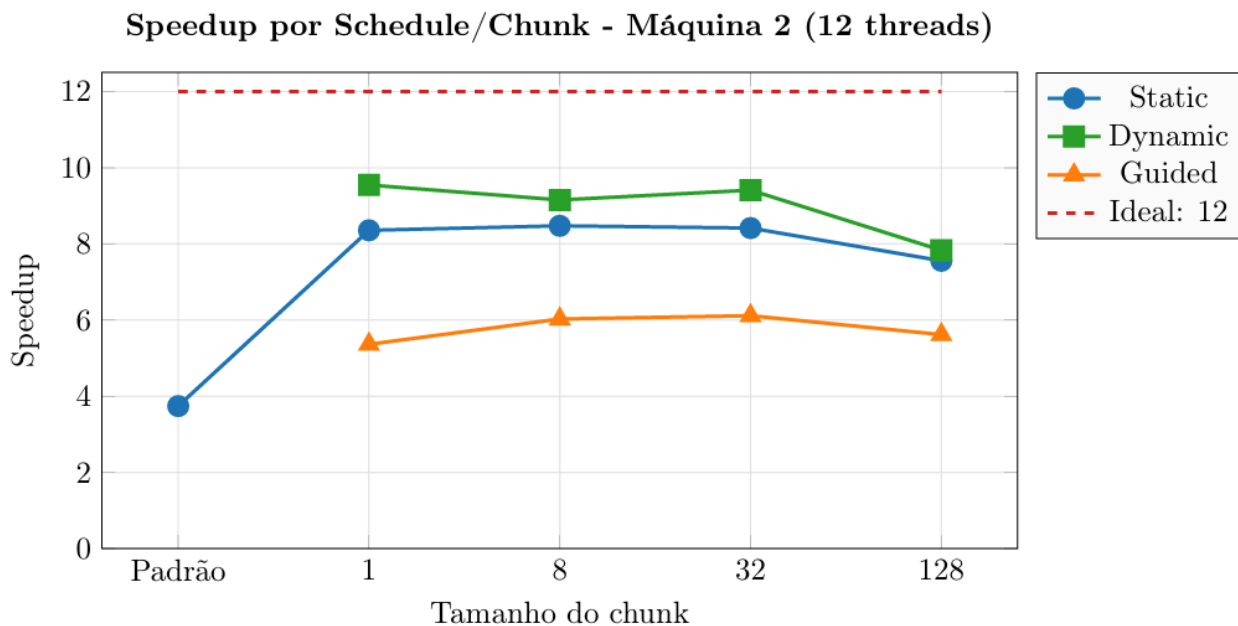


Figura 9 – Speedup obtido para as diferentes políticas de escalonamento na Máquina 2. O Speedup foi calculado com base no tempo sequencial de referência 6.84s.

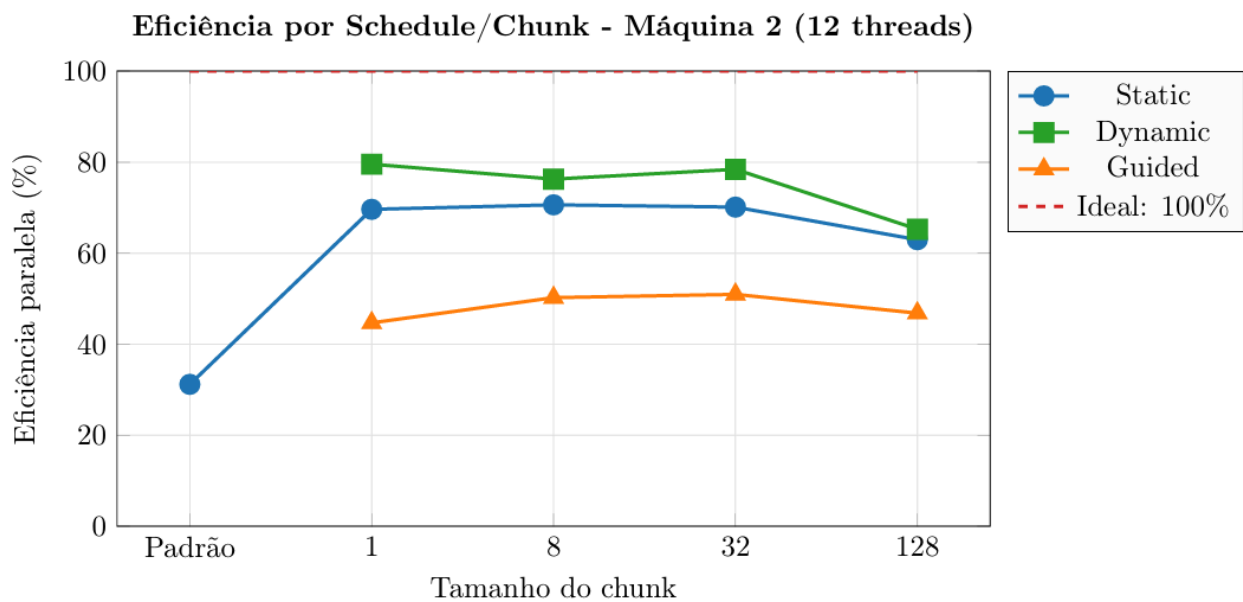


Figura 10 – Eficiência Paralela obtida para as diferentes políticas de escalonamento na Máquina 2.

Carga mínima, média e máxima

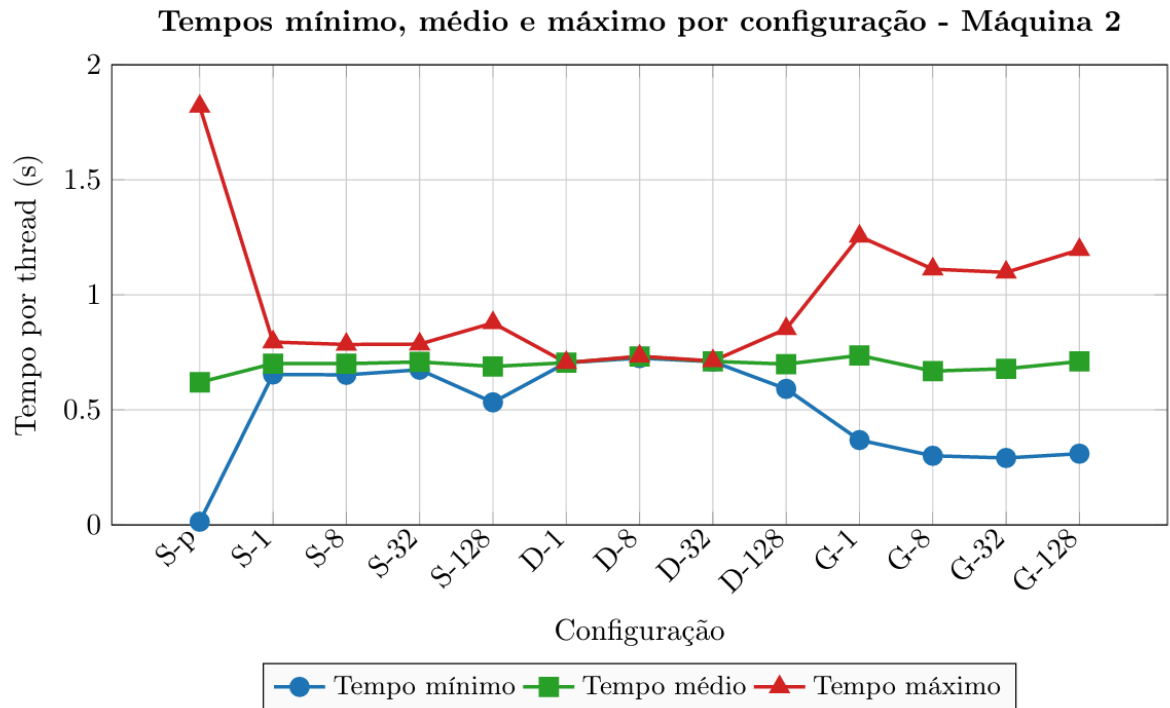


Figura 11 – Cargas obtidas para as diferentes políticas de escalonamento na Máquina 2.

Fator de Balanceamento de Carga

Fator de Balanceamento por Schedule/Chunk - Máquina 2 (12 threads)

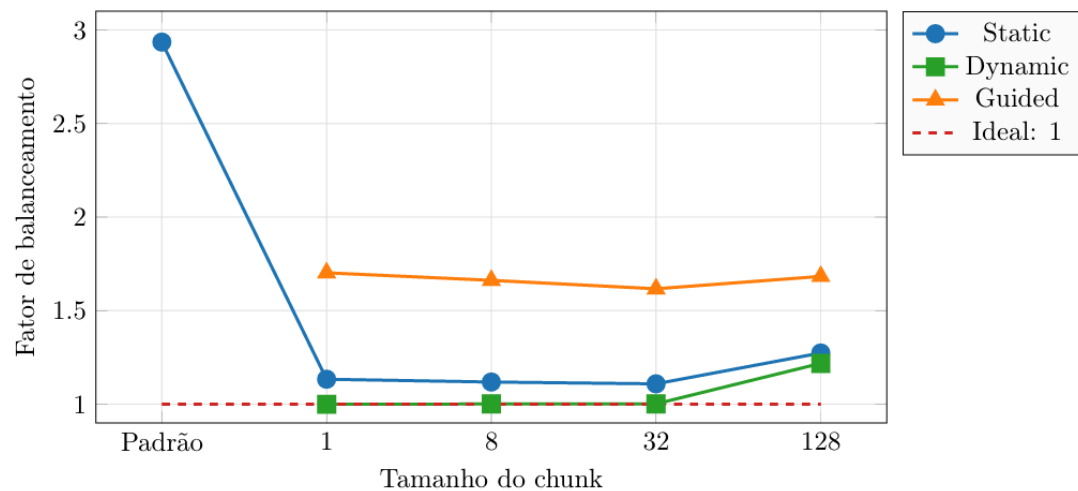


Figura 12 – Fator de Balanceamento de Carga obtido para as diferentes políticas de escalonamento na Máquina 2.

3.4 Casos de Estresse

3.4.1 Escalabilidade Forte (Casos de Teste)

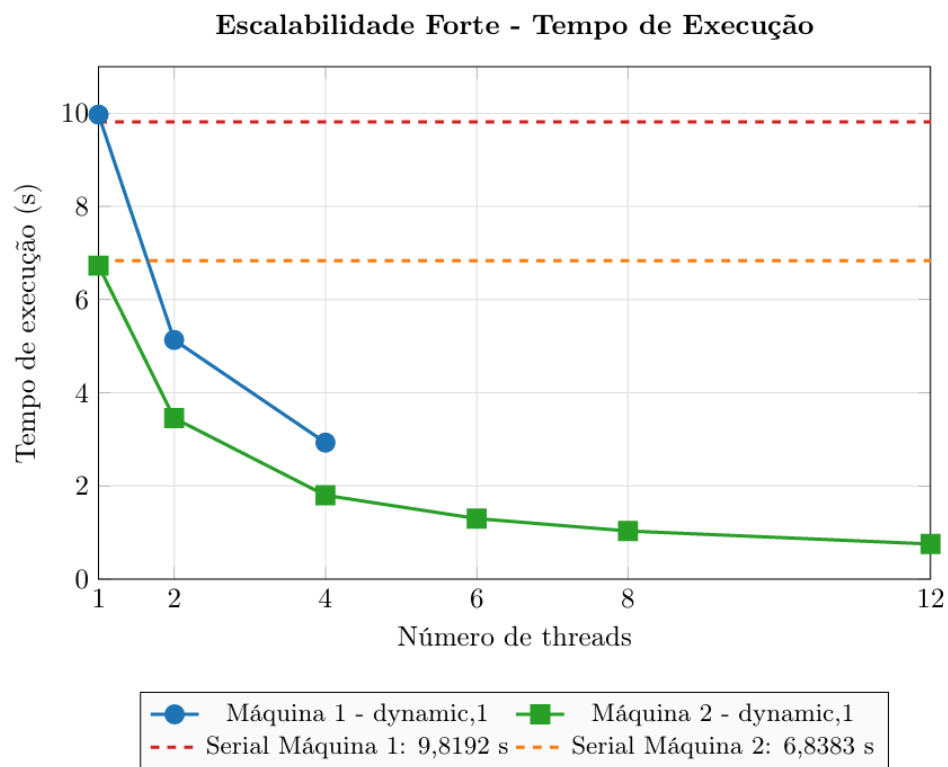
Tabela 3: Resultados de escalabilidade forte na Máquina 1, utilizando a melhor configuração observada nos testes anteriores (*dynamic,1*) e resolução fixa de 4096x4096, *MAX_ITER* = 1000.

Threads	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
1	9.972087	9.819213	0.985	98.47	1.000000
2	5.134144	9.819214	1.913	95.63	1.000000
4	2.930167	9.819215	3.351	83.78	1.000348

Tabela 4: Resultados de escalabilidade forte na Máquina 2, utilizando a melhor configuração observada nos testes anteriores (*dynamic,1*) e resolução fixa de 4096x4096, *MAX_ITER* = 1000.

Threads	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
1	6.730127	6.838324	1.016	101.61	1.000000
2	3.455442	6.838324	1.979	98.95	1.000000
4	1.798214	6.838324	3.803	95.07	1.000023
6	1.298747	6.838324	5.265	87.76	1.000018
8	1.032180	6.838324	6.625	82.81	1.000168
12	0.752160	6.838324	9.092	75.76	1.003147

3.4.1.1 Representação Gráfica



*Figura 13 – Tempo de Execução na análise de escalabilidade forte
utilizando a configuração dynamic,1.*

Speedup e Eficiência

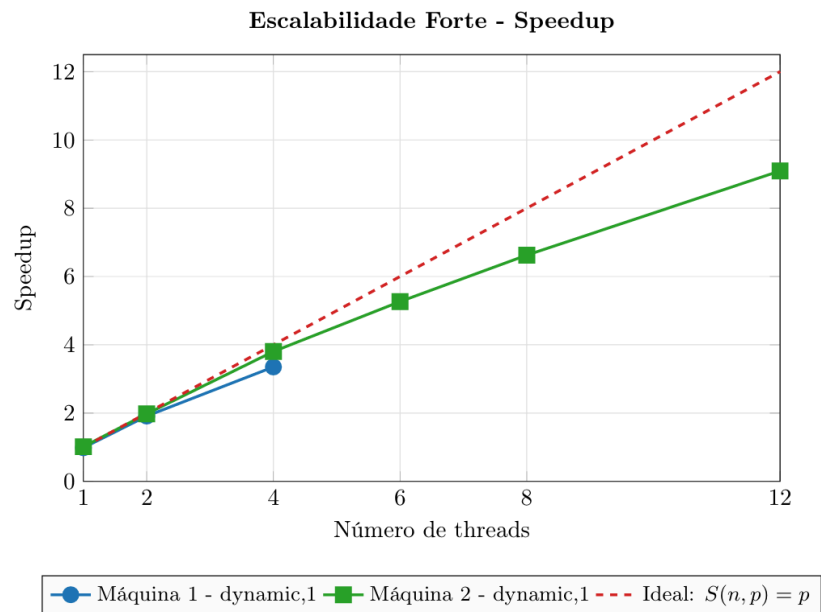


Figura 14 – Speedup obtido na análise da escalabilidade forte. O Speedup foi calculado com base no tempo sequencial de referência de cada máquina.

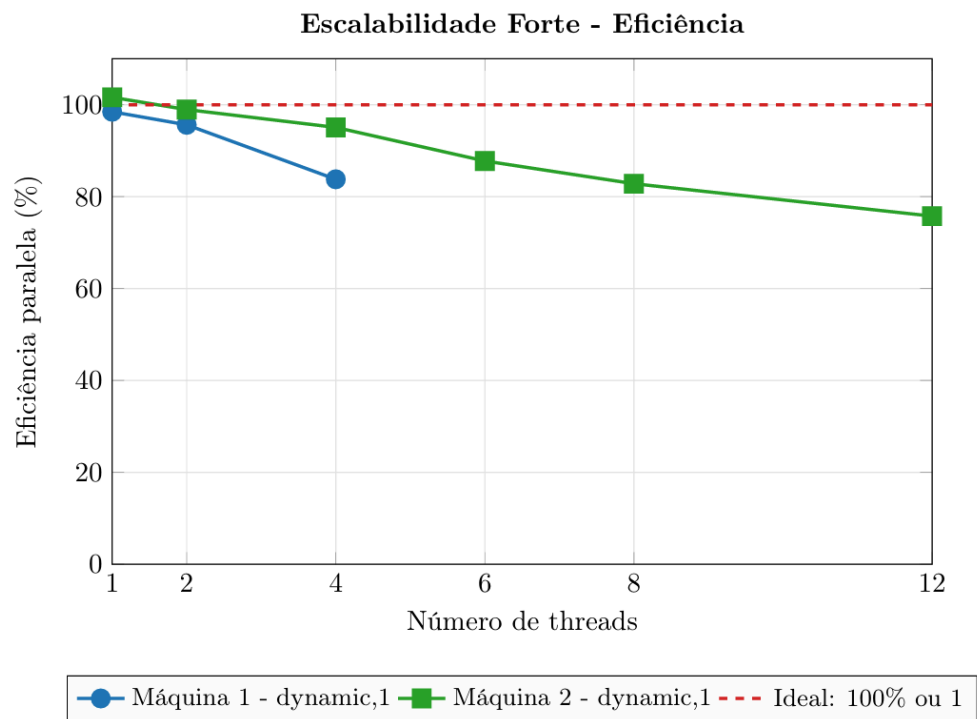


Figura 15 – Eficiência paralela obtida na análise da escalabilidade forte.

Carga mínima, média e máxima

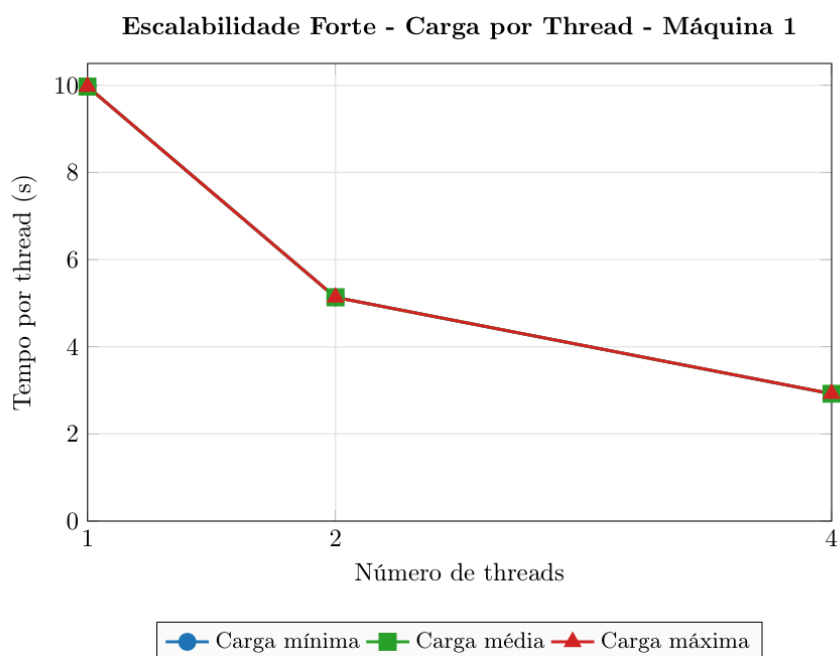


Figura 16 – Distribuição das cargas mínima, média e máxima por thread durante a análise de escalabilidade forte da Máquina 1.

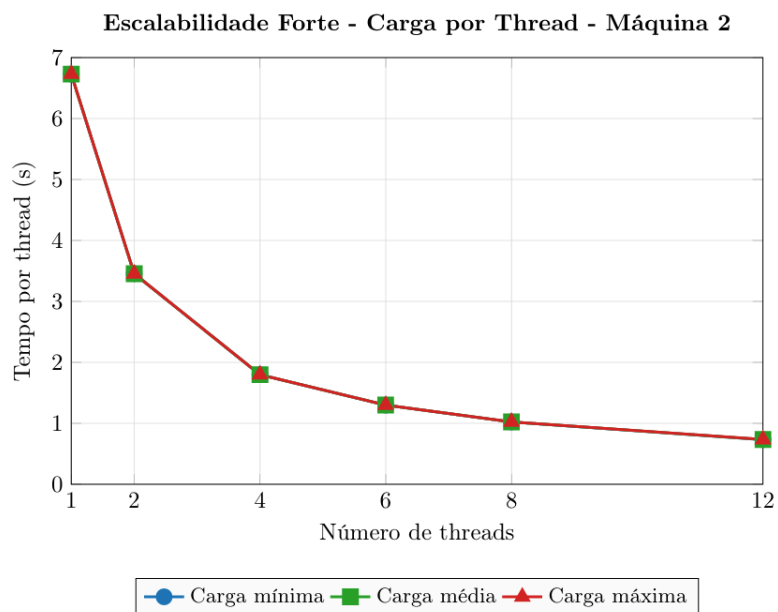


Figura 17 – Distribuição das cargas mínima, média e máxima por thread durante a análise de escalabilidade forte da Máquina 2.

Fator de Balanceamento de Carga

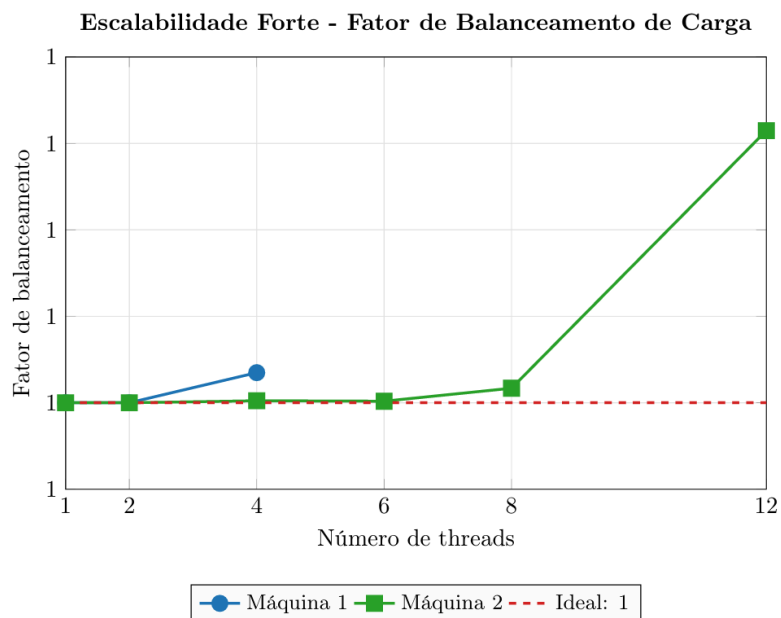


Figura 18 – Fator de balanceamento de carga obtido na análise de escalabilidade forte.

Valores mais próximos de 1 indicam melhor distribuição de cargas entre as threads.

3.4.2 Desbalanceamento (Casos de Teste)

3.4.2.1 Representação Gráfica

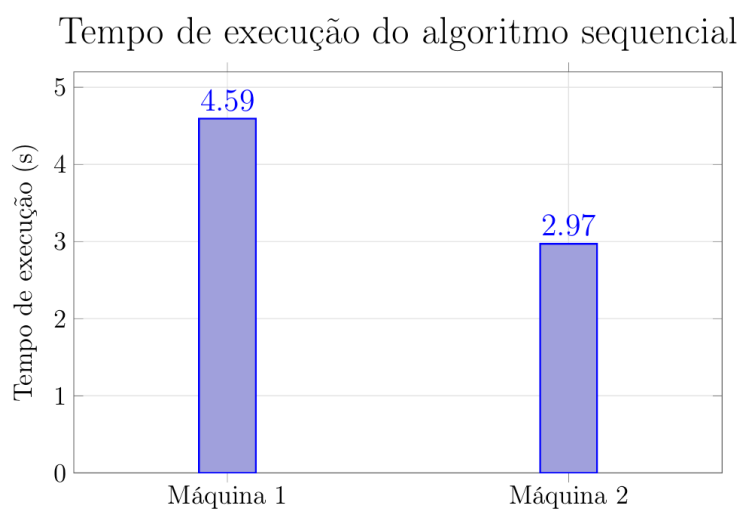


Figura 19 – Comparação do Tempo de Execução da versão sequencial no caso de desbalanceamento, utilizando resolução 4096x4096 e MAX_ITER = 5000.

Tabela 5: Resultados dos testes de desbalanceamento na Máquina 1, utilizando resolução 4096x4096, MAX_ITER = 5000 e 4 threads.

Schedule	Chunk	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
static	padrão	1.567074	4.593585	2.931	73.28	1.212017
static	1	1.344861	4.593586	3.416	85.39	1.023786
static	8	1.337393	4.593587	3.435	85.87	1.010785
static	32	1.337393	4.593588	3.435	85.87	1.010785
static	128	1.421732	4.593589	3.231	80.77	1.030688
dynamic	1	1.410053	4.593590	3.258	81.44	1.001018
dynamic	8	1.388198	4.593591	3.309	82.73	1.007985
dynamic	32	1.286538	4.593592	3.571	89.26	1.015777
dynamic	128	2.158690	4.593593	2.128	53.20	1.055472
guided	1	1.489187	4.593594	3.085	77.12	1.104182
guided	8	1.644689	4.593595	2.793	69.82	1.097363
guided	32	1.424902	4.593596	3.224	80.59	1.111715
guided	128	1.693179	4.593597	2.713	67.83	1.159934

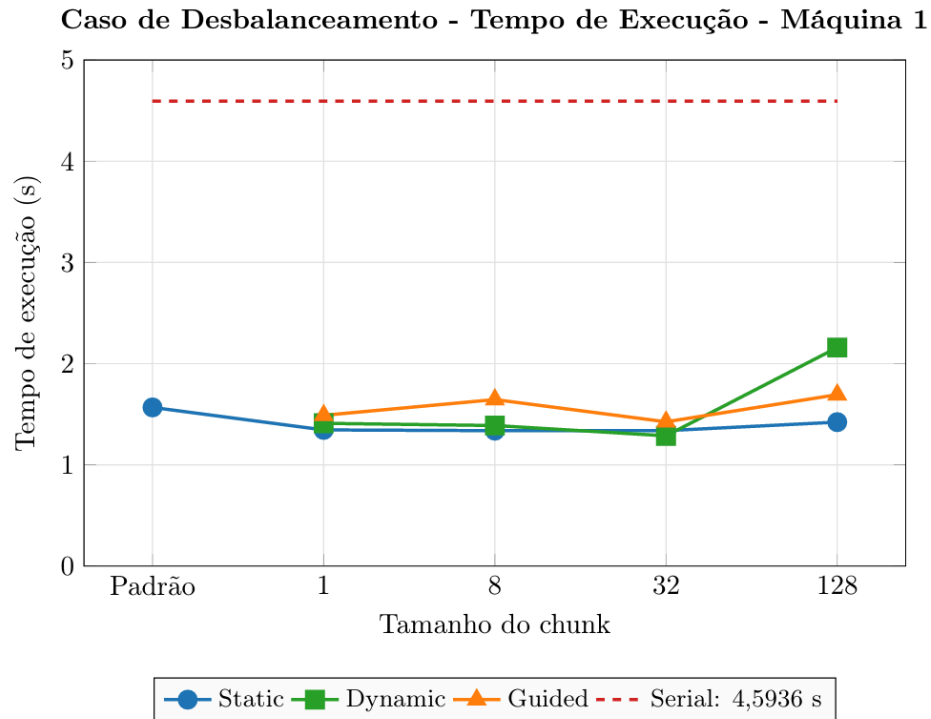


Figura 20 – Tempo de Execução das políticas de escalonamento no caso de desbalanceamento para a Máquina 1. A linha vermelha tracejada representa o tempo da versão sequencial de referência.

Speedup e Eficiência

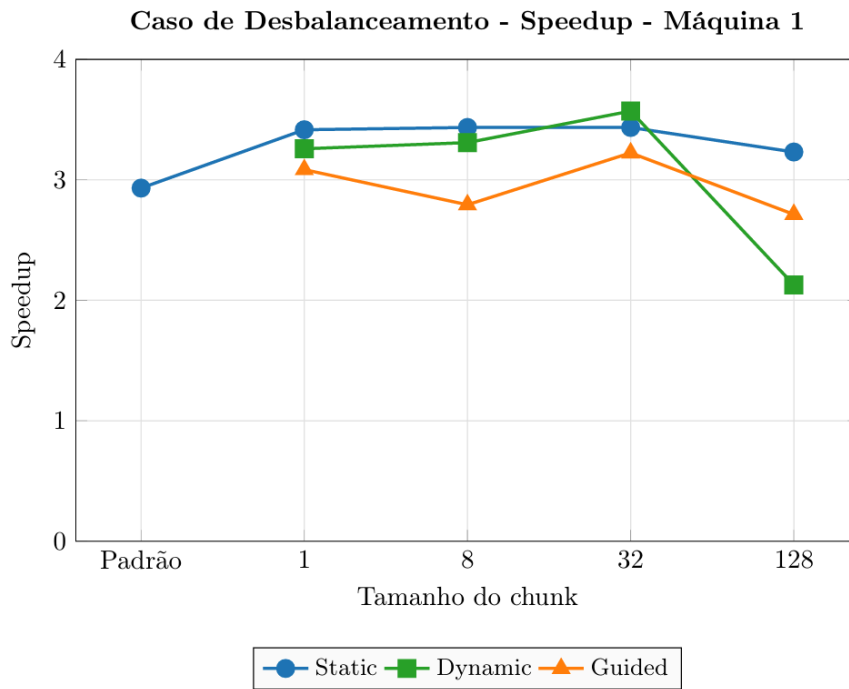


Figura 21 – Speedup obtido pelas diferentes políticas de escalonamento no caso de desbalanceamento para a Máquina 1. O Speedup foi calculado no tempo sequencial de referência da própria máquina.

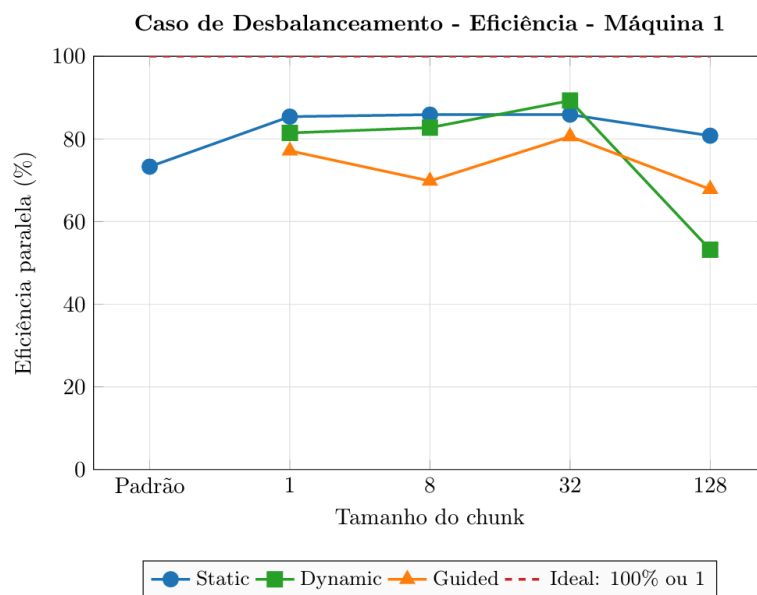


Figura 22 – Eficiência obtida pelas diferentes políticas de escalonamento no caso de desbalanceamento para a Máquina 1.

Carga mínima, média e máxima

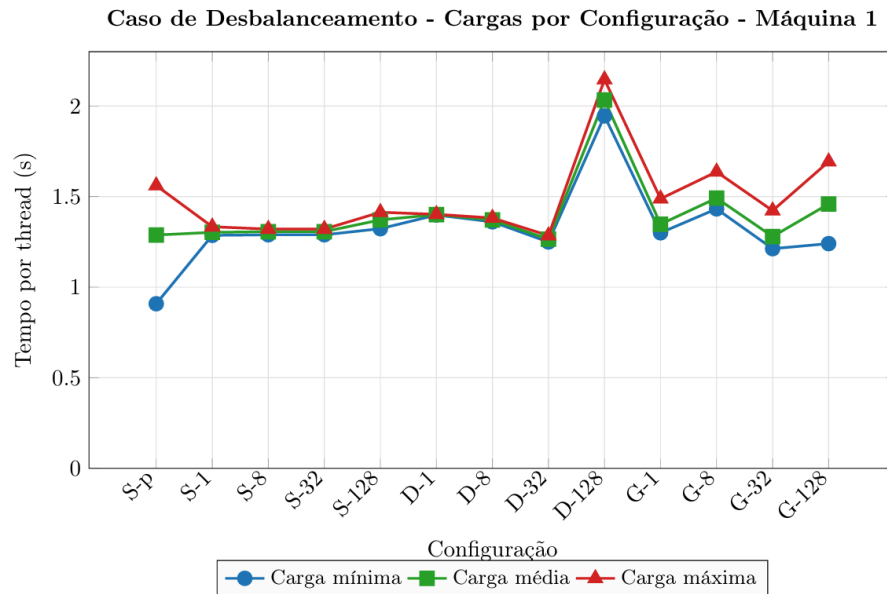


Figura 23 – Distribuição das cargas mínima, média e máxima por thread no caso de desbalanceamento para a Máquina 1. Quanto mais próximas estiverem as três curvas, melhor é o balanceamento de carga.

Fator de Balanceamento de Carga

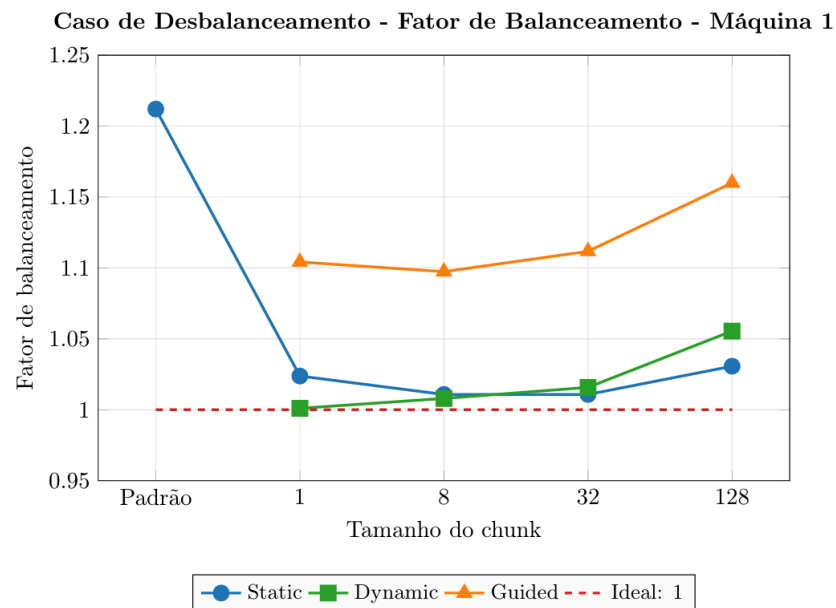


Figura 24 – Fator de balanceamento de carga no caso de desbalanceamento para a Máquina 1. Valores mais próximos de 1 indicam melhor distribuição de carga.

Tabela 6: Resultados dos testes de desbalanceamento na Máquina 2, utilizando resolução 4096x4096, MAX_ITER = 5000 e 12 threads.

Schedule	Chunk	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
static	padrão	1.856670	2.970497	1.600	13.33	2.957839
static	1	0.813193	2.970497	3.653	30.44	1.114208
static	8	0.827017	2.970497	3.592	29.93	1.118596
static	32	0.792780	2.970497	3.747	31.22	1.093551
static	128	0.805872	2.970497	3.686	30.72	1.110261
dynamic	1	0.745882	2.970497	3.983	33.19	1.004283
dynamic	8	0.744091	2.970497	3.992	33.27	1.000465
dynamic	32	0.831357	2.970497	3.573	29.78	1.056577
dynamic	128	0.926697	2.970497	3.205	26.71	1.254122
guided	1	1.131604	2.970497	2.625	21.88	1.624793
guided	8	1.177599	2.970497	2.523	21.02	1.713927
guided	32	1.177276	2.970497	2.523	21.03	1.720208
guided	128	1.148048	2.970497	2.587	21.56	1.651653

Caso de Desbalanceamento - Tempo de Execução - Máquina 2

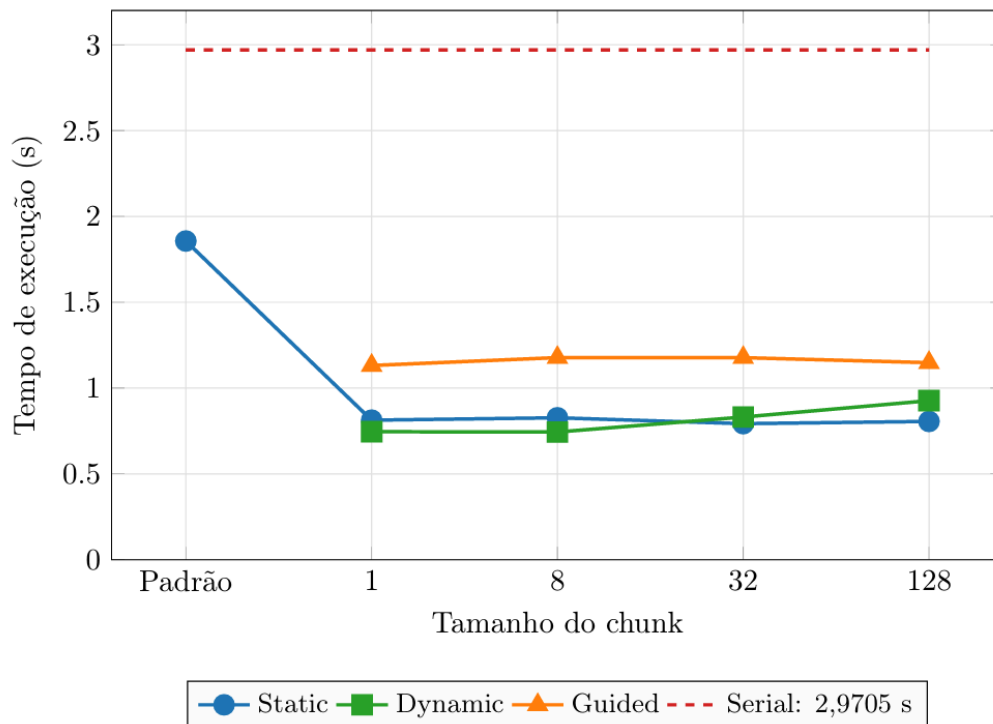


Figura 25 – Tempo de Execução das políticas de escalonamento no caso de desbalanceamento para a Máquina 2. A linha vermelha tracejada representa o tempo da versão sequencial de referência.

Speedup e Eficiência

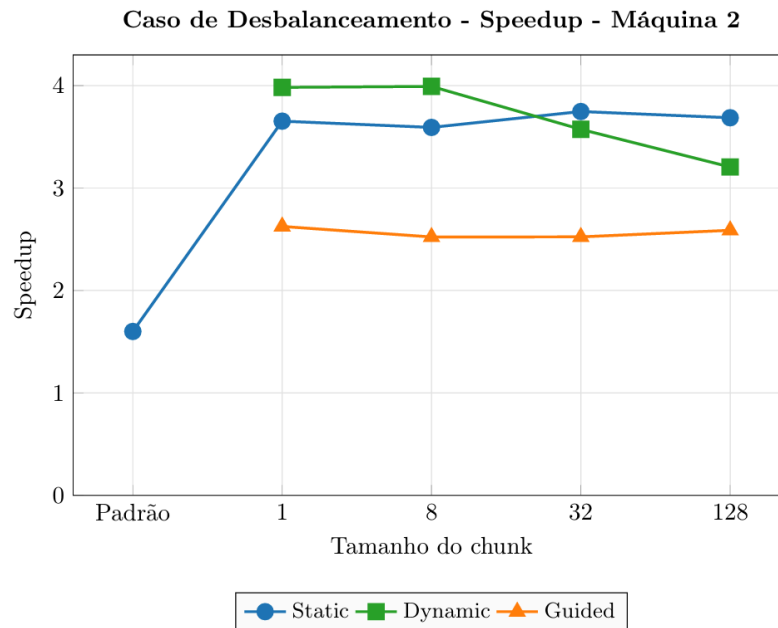


Figura 26 – Speedup obtido pelas diferentes políticas de escalonamento no caso de desbalanceamento para a Máquina 2. O Speedup foi calculado no tempo sequencial de referência da própria máquina.

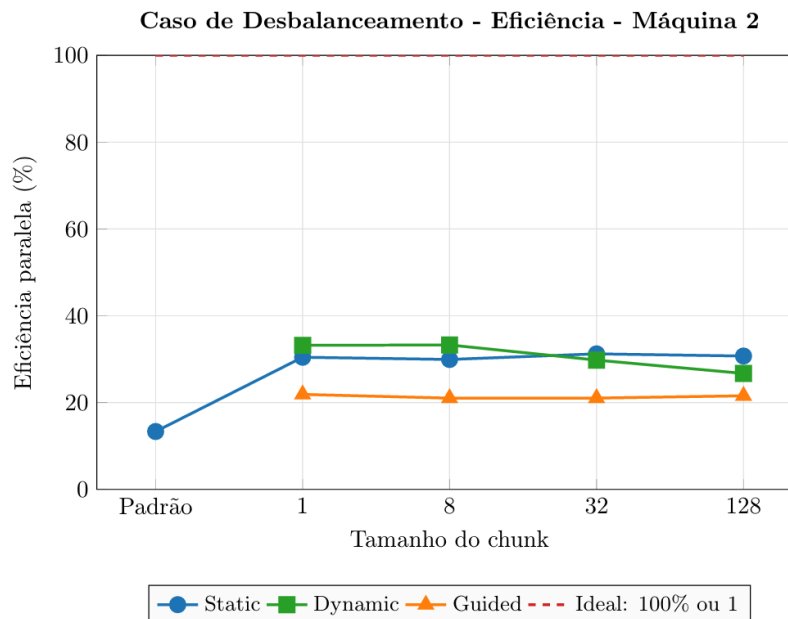


Figura 27 – Eficiência obtida pelas diferentes políticas de escalonamento no caso de desbalanceamento para a Máquina 2.

Carga mínima, média e máxima

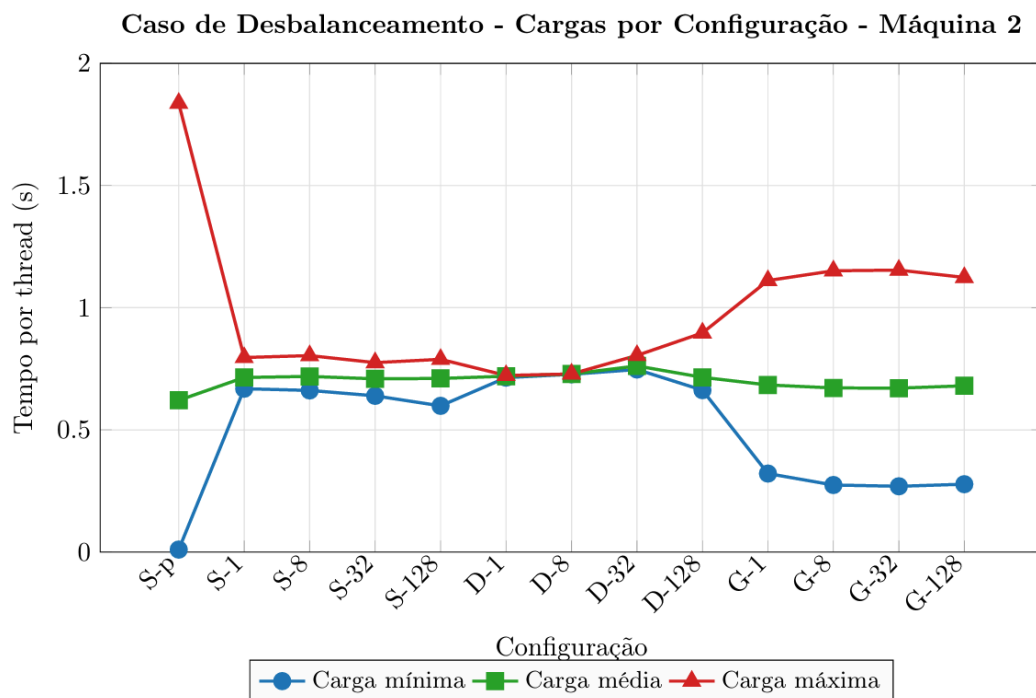


Figura 28 – Distribuição das cargas mínima, média e máxima por thread no caso de desbalanceamento para a Máquina 2. Quanto mais próximas estiverem as três curvas, melhor é o balanceamento de carga.

Fator de Balanceamento de Carga

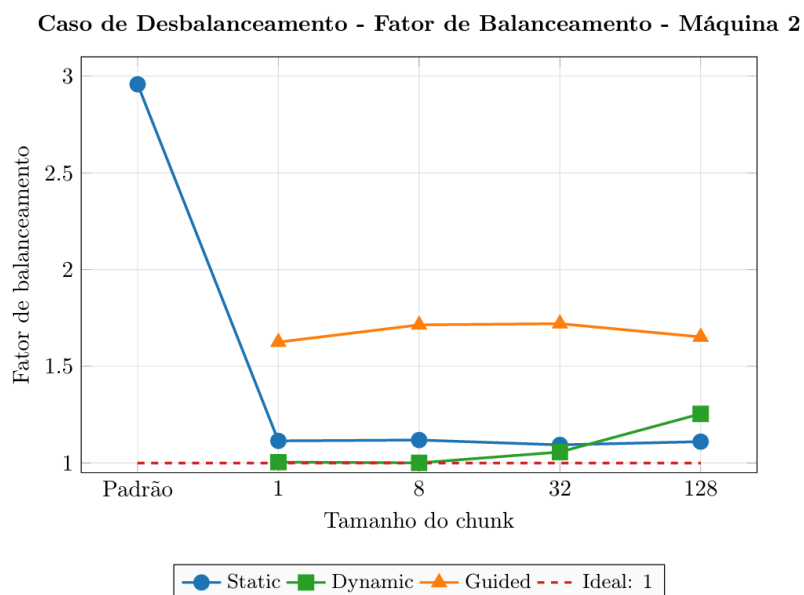


Figura 29 – Fator de balanceamento de carga no caso de desbalanceamento para a Máquina 2. Valores mais próximos de 1 indicam melhor distribuição de carga.

3.4.3 Escalabilidade Fraca (Casos de Teste)

3.4.3.1 Representação Gráfica

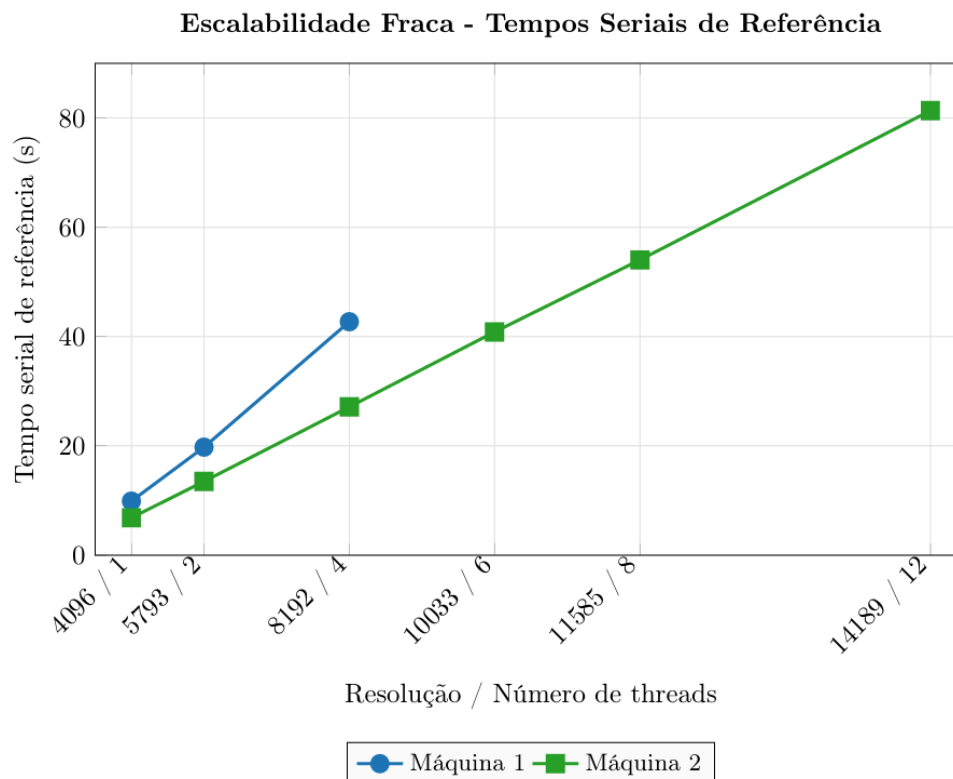


Figura 30 – Tempos seriais de referência utilizados nos experimentos de escalabilidade fraca. Cada ponto do eixo horizontal representa a combinação entre resolução e números de threads considerada no experimento.

Tabela 7: Resultados de escalabilidade fraca na Máquina 1, utilizando a melhor configuração observada nos testes anteriores (*dynamic,1*), variando as resoluções, *MAX_ITER = 1000* e variando as threads (1,2,4)

Resolução	Threads	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
4096	1	10.214146	9.886168	0.968	96.79	1.000000
5793	2	10.311322	19.761968	1.917	95.83	1.000023
8192	4	11.865010	42.719269	3.600	90.01	1.000039

Tabela 8: Resultados de escalabilidade fraca na Máquina 2, utilizando a melhor configuração observada nos testes anteriores (*dynamic,1*), variando as resoluções, *MAX_ITER* = 1000 e variando as threads (1,2,4,6,8,12)

Resolução	Threads	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
4096	1	6.722251	6.838324	1.017	101.73	1.000000
5793	2	6.886575	13.489908	1.959	97.94	1.000001
8192	4	7.297215	27.133436	3.718	92.96	1.000001
10033	6	7.895041	40.851994	5.174	86.24	1.000004
11585	8	8.600772	54.031617	6.282	78.53	1.000051
14189	12	9.942284	81.347114	8.182	68.18	1.000064

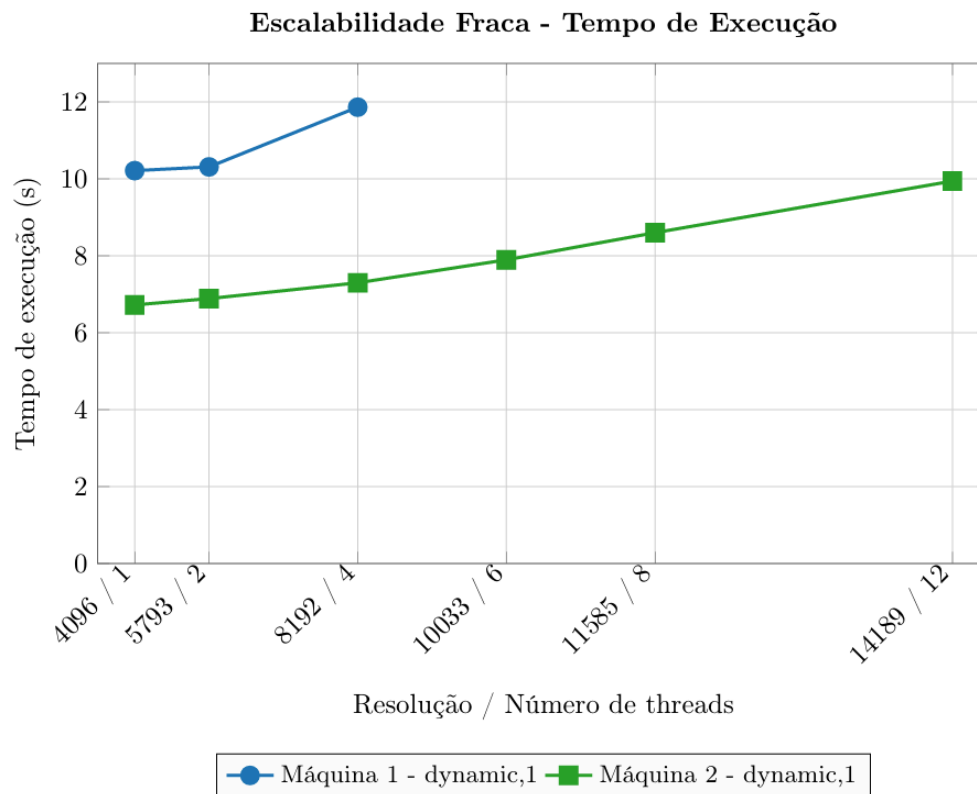


Figura 31 – Tempo de execução obtido nos experimentos de escalabilidade fraca utilizando a configuração *dynamic,1*. O tamanho do problema foi aumentado juntamente com o número

de threads, buscando manter aproximadamente constante a quantidade de trabalho por unidade de processamento.

Speedup e Eficiência

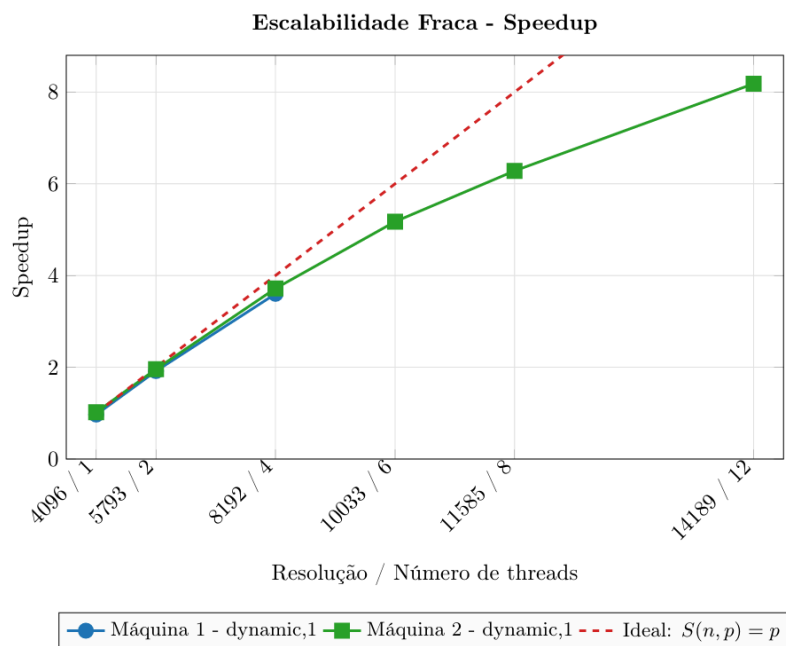


Figura 32 – Speedup obtido nos experimentos de escalabilidade fraca, utilizando a configuração dynamic,1. Cada ponto foi calculado a partir do tempo serial de referência da mesma resolução correspondente ao número de threads.

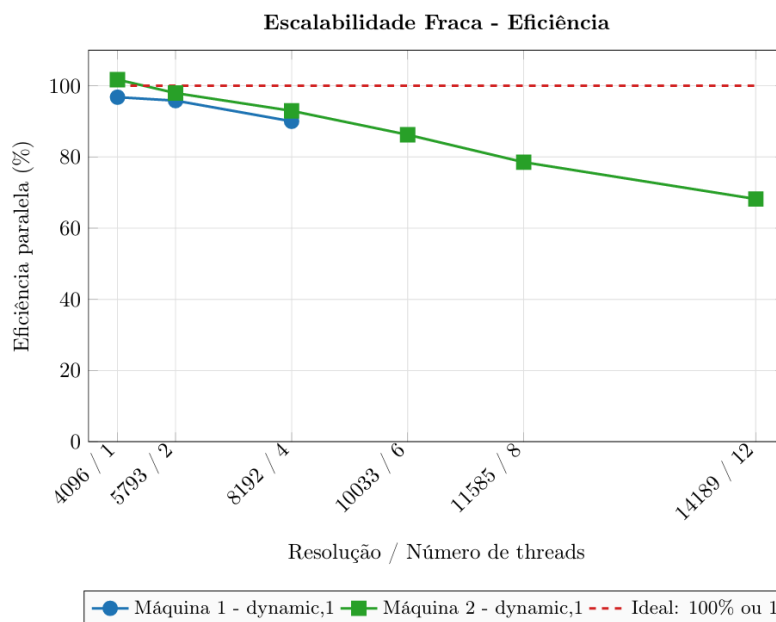


Figura 33 – Eficiência paralela obtida nos experimentos de escalabilidade fraca, utilizando a configuração dynamic,1.

Carga mínima, média e máxima

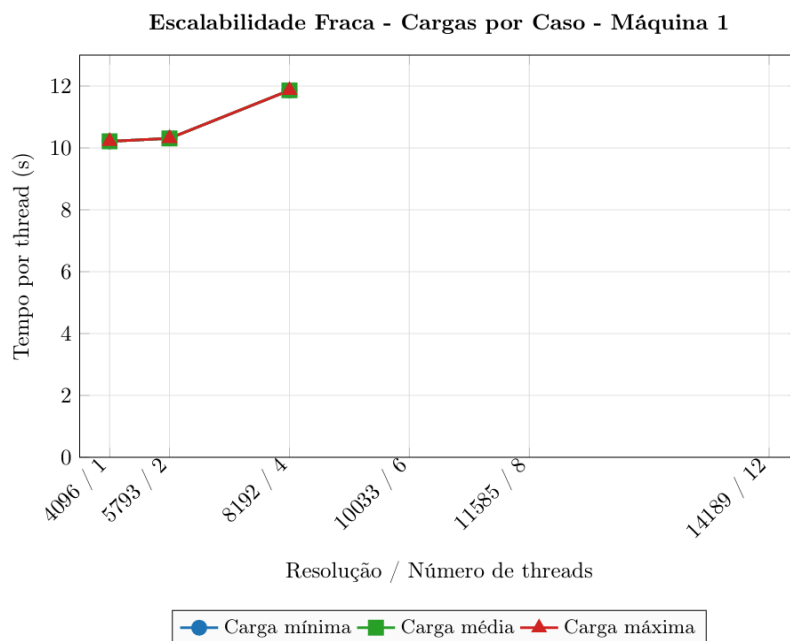


Figura 34 – Distribuição das cargas mínima, média e máxima por thread no caso de escalabilidade fraca para a Máquina 1.

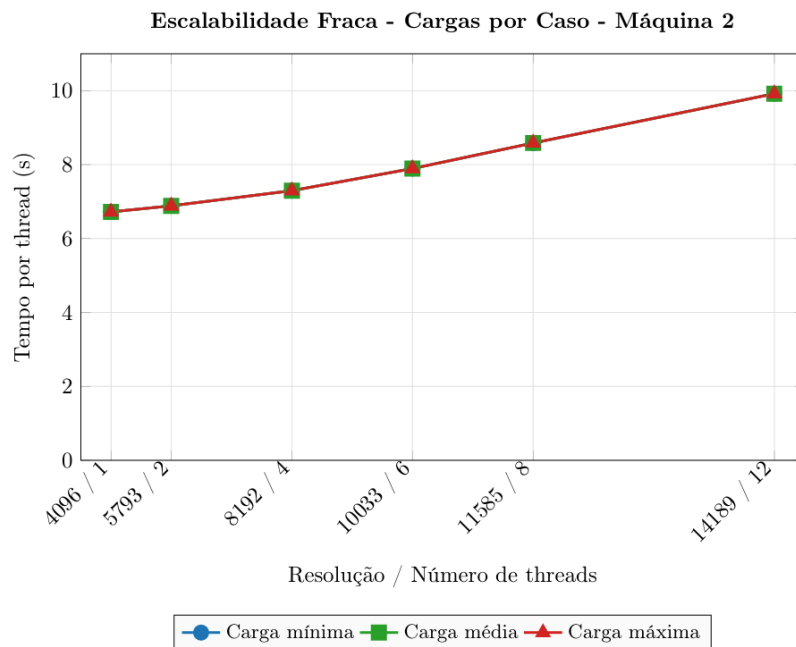


Figura 35 – Distribuição das cargas mínima, média e máxima por thread no caso de escalabilidade fraca para a Máquina 2.

Fator de Balanceamento de Carga

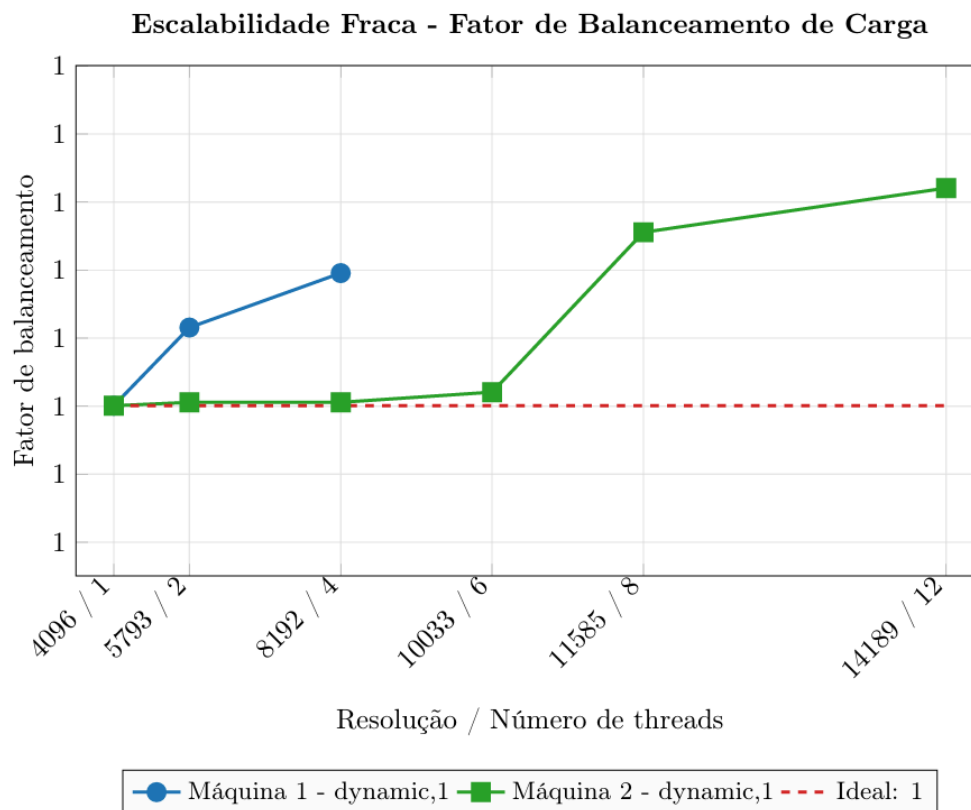


Figura 36 – Fator de balanceamento de carga obtido nos experimentos de escalabilidade fraca. Cada ponto representa uma combinação entre resolução e número de threads. Valores próximos de 1 indicam uma distribuição equilibrada da carga computacional entre as threads.

3.4.5 Comparação entre máquinas (1 e 2)

3.4.5.1 Representação Gráfica

Tabela 9: Comparação de desempenho entre as duas máquinas (1 e 2), utilizando 4 threads e a configuração *dynamic,1*.

Máquina	Threads	Tempo (s)	Tempo serial ref. (s)	Speedup	Eficiência (%)	Fator de Balanceamento
Máquina 1	4	3.708679	9.901376	2.670	66.74	1.000289
Máquina 2	4	1.792740	6.838324	3.814	95.36	1.000008

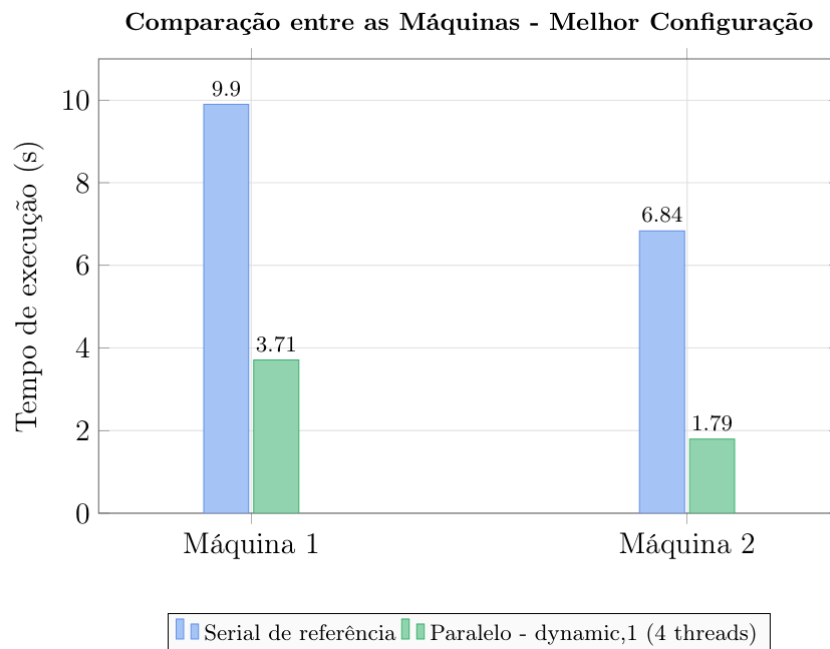


Figura 37 – Comparação entre as máquinas 1 e 2. Ambas as execuções paralelas utilizaram 4 threads, resolução 4096x4096, MAX_ITER=1000 e a configuração dynamic,1. O tempo serial de referência de cada máquina é apresentado para comparação.

Speedup e Eficiência

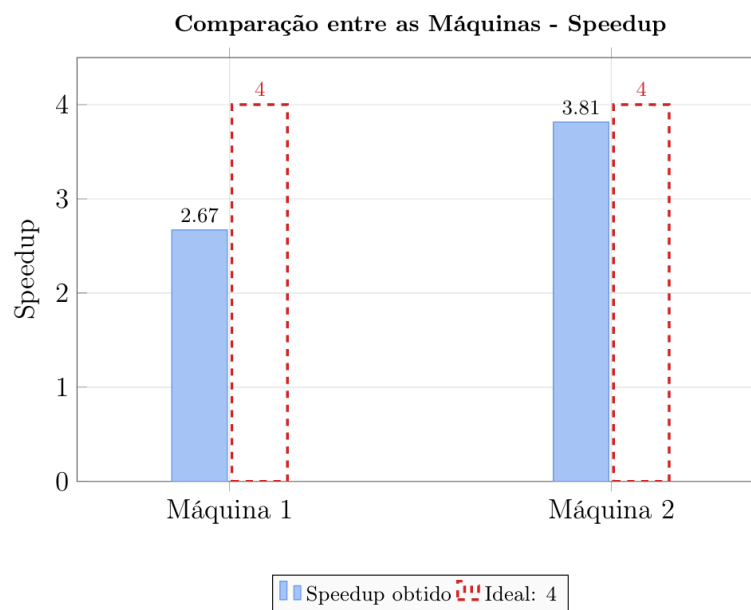


Figura 38 – Speedup das máquinas. A linha tracejada indica o speedup ideal para 4 threads.

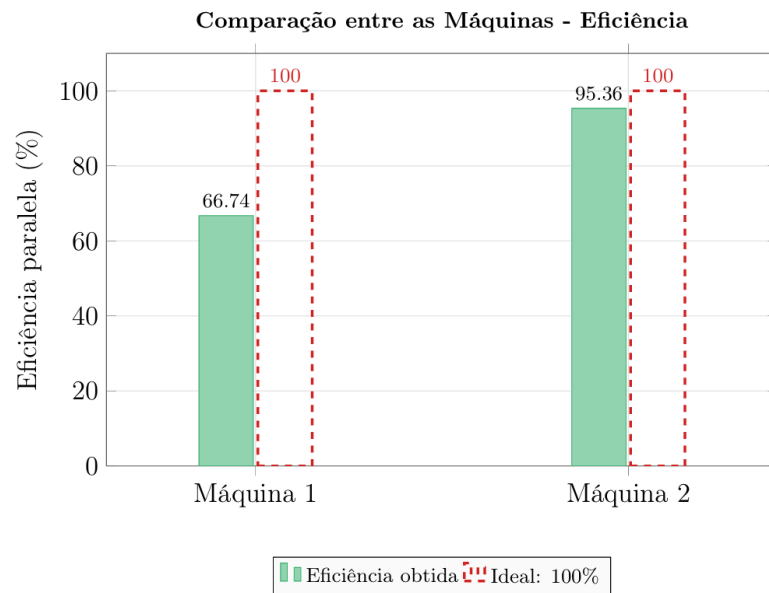


Figura 39 – Eficiência das máquinas. A linha tracejada indica a eficiência ideal.

Carga mínima, média e máxima

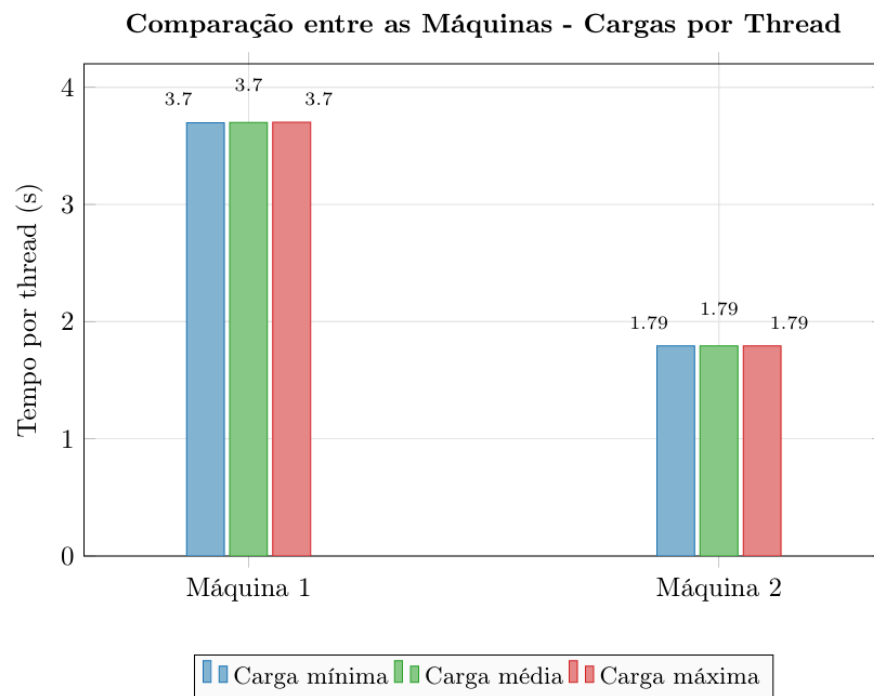


Figura 40 – Comparação das cargas mínima, média e máxima por thread entre as máquinas.

Fator de Balanceamento de Carga

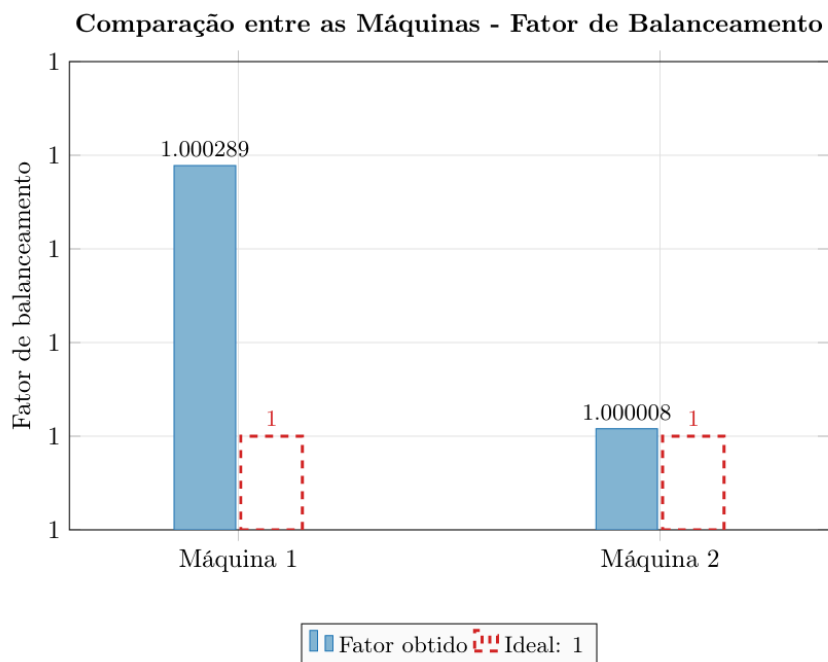


Figura 41 – Comparação do fator de balanceamento de carga entre as duas máquinas. Ambas utilizaram a configuração *dynamic,1* com 4 threads. Valores próximos de 1 indicam melhor equilíbrio da carga entre as threads.

4. Discussão

4.1 Análise da Corretude da Implementação

Como o cálculo do Conjunto de Mandelbrot é inerentemente livre de dependências de dados, a integridade estrutural da matriz final confirma que a gravação concorrente no espaço de memória, governada pelo indexamento determinístico do mapeamento row-major, ocorreu sem colisões estruturais ou condições de corrida. Esse comportamento atesta que o particionamento do laço externo pelo OpenMP garantiu a segurança da concorrência de forma natural, dispensando a introdução de dispendiosos mecanismos de exclusão mútua, como zonas críticas ou travas atômicas, e preservando o determinismo matemático estrito estabelecido pela versão de referência.

4.2 Comparação entre a Versão Sequencial e a Versão Paralela em OpenMP

O contraste de desempenho entre as abordagens evidencia a forte aderência do problema de Mandelbrot à aceleração em arquiteturas multicore. A execução sequencial atua como um gargalo clássico por restringir-se ao processamento linear iterativo de cada coordenada do plano, o que, conforme ilustrado na Figura 2, exigiu um tempo base de 9,82 segundos na Máquina 1 e 6,84 segundos na Máquina 2 para renderizar a resolução padrão.

A transição para a paralelização transformou o comportamento do algoritmo ao descentralizar a sobrecarga espacial. Na Máquina 1, cuja topologia restringe-se a 2 núcleos físicos com suporte a Hyper-Threading, a injeção da política *dynamic* com *chunk 1* mitigou o tempo computacional para aproximadamente 2,80 segundos, um ganho diretamente evidenciado na Figura 3. Este avanço comprova que a arquitetura paralela permitiu saturar eficientemente as unidades lógicas disponíveis, contornando a limitação física do processador e alcançando um *Speedup* próximo ao teto arquitetural da máquina, como demonstrado na Figura 4.

A disparidade de ganho revela-se ainda mais categórica na arquitetura híbrida de 12 threads da Máquina 2. Conforme exposto na Figura 8, a distribuição de carga reduziu drasticamente o tempo de execução para 0,71 segundos sob o escalonamento *dynamic*, 1. Esse resultado atesta que o particionamento dinâmico em memória compartilhada viabiliza a orquestração simultânea de núcleos de alta performance e núcleos de eficiência, traduzindo o esforço paralelo em acelerações massivas e consolidadas no expressivo *Speedup* detalhado pela Figura 9.

4.3 Análise do Impacto do Número de Threads no Desempenho

A avaliação da escalabilidade forte, cujo propósito é medir o ganho de desempenho mantendo o tamanho do problema fixo enquanto se amplia o número de processadores, revelou o comportamento clássico modelado pela Lei de Amdahl. Conforme ilustrado na Figura 13, o tempo de execução apresenta uma queda acentuada e contínua com a adição de threads, demonstrando a eficácia da paralelização na distribuição do esforço

computacional. Contudo, observa-se que a taxa de redução temporal torna-se sublinear à medida que o grau de concorrência aumenta.

A Figura 14 evidencia que o Speedup reflete estritamente essa tendência: embora a aceleração do algoritmo seja progressiva, a curva empírica desvia-se de forma gradativa da linha de aceleração ideal $S(n) = p$. Este fenômeno é quantificado pela métrica de Eficiência Paralela, exposta na Figura 15, que registra uma degradação assintótica natural. Na Máquina 1, a eficiência inicia próxima a 100% e decai para aproximadamente 83% ao utilizar a capacidade máxima de 4 threads lógicas. Na Máquina 2, sob maior grau de paralelismo, a eficiência encerra em cerca de 75% ao saturar o sistema com 12 threads.

Essa perda fracional de eficiência justifica-se por limitadores teóricos e arquiteturais. Primeiramente, a porção inerentemente sequencial do programa e o overhead introduzido pela infraestrutura do OpenMP, que abrange a criação do grupo de threads, o gerenciamento de requisições de carga da política *dynamic* e a sincronização em barreiras implícitas, tornam-se proporcionalmente mais expressivos conforme o tempo de processamento útil de cada thread é encurtado. Em segundo lugar, a contenção por recursos físicos do hardware impõe um teto ao escalonamento; o compartilhamento de Unidades Lógicas e Aritméticas (ALUs) em núcleos operando via Hyper-Threading, somado ao gargalo gerado pela concorrência no barramento de acesso à memória RAM para a gravação da matriz, impedem que a escalabilidade atinja a linearidade perfeita.

4.4 Discussão Fundamentada sobre o Balanceamento de Carga

A arquitetura do algoritmo de escape-time para o Conjunto de Mandelbrot apresenta uma assimetria computacional inerente, uma vez que o esforço iterativo varia drasticamente consoante a coordenada espacial. No cenário de estresse, focado na região caótica do "vale dos cavalos-marinhos" com um limite elevado de 5000 iterações, esta disparidade atinge o seu expoente máximo. Grande parte dos píxeis nesta zona escapa à condição de divergência em iterações iniciais, enquanto uma fronteira restrita e complexa retém o processamento até ao limite estipulado. Este padrão geométrico cria concentrações localizadas de carga, tornando o particionamento equitativo do trabalho um desafio crítico para o desempenho.

A análise empírica demonstra que a política estática padrão (*static*) falha severamente perante este desbalanceamento acentuado. Ao fatiar o laço em grandes blocos contíguos de forma determinística, esta estratégia confina a região mais densa do fractal a uma minoria de threads, enquanto as restantes esgotam as suas áreas de escape rápido numa fração de segundo e permanecem ociosas em barreiras de sincronização. Tal facto é inequivocamente exposto na Máquina 2, onde o escalonamento estático padrão registou um Tempo de Execução de 1,85 segundos e uma Eficiência Paralela de apenas 13,33% (Tabela 6). A Figura 28 ilustra graficamente este colapso: a curva de carga máxima distancia-se de forma abissal das curvas de carga média e mínima, resultando num Fator de Balanceamento de 2,95 (Tabela 6 e

Figura 29), o que denota uma gravíssima ineficiência e ociosidade sistêmica. Contudo, a introdução de uma granularidade fina (static, 1) mitiga este cenário ao distribuir as linhas da matriz de forma intercalada, melhorando a distribuição geométrica do trabalho, reduzindo o fator de balanceamento para 1,11 e o tempo para 0,81 segundos nesta mesma máquina (Tabela 6).

Apesar da melhoria com fragmentos menores, foi a política dynamic que se revelou a estratégia mais eficaz para suplantiar o desbalanceamento espacial. Ao adotar uma distribuição sob demanda, as threads que finalizavam as zonas menos densas requisitavam prontamente novos blocos de processamento em tempo de execução. Este comportamento dinâmico equalizou o esforço global da aplicação, facto comprovado visualmente pelas Figuras 23 e 28, onde as curvas de tempo mínimo, médio e máximo da configuração dynamic, 1 praticamente se sobrepõem. O Fator de Balanceamento convergiu para o patamar ótimo, registrando valores em torno de 1,00 em ambas as máquinas avaliadas (Tabela 5 e Tabela 6). Como consequência da eliminação quase total do tempo de espera, esta configuração alcançou os melhores tempos absolutos da bateria de estresse, totalizando 0,74 segundos na Máquina 2 e sustentando um expressivo Speedup de 3,98 (Tabela 6 e Figura 26).

Por sua vez, a política guided apresentou um desempenho inferior ao esperado para este cenário. Concebida com o intuito de atenuar o overhead diminuindo progressivamente o tamanho dos blocos, os seus amplos chunks iniciais acabaram por capturar de forma precoce as zonas mais densas do fractal. Esta alocação inicial enviesou a carga de trabalho logo no arranque da rotina paralela, inviabilizando uma redistribuição eficaz nas iterações remanescentes. Observa-se que os Fatores de Balanceamento da configuração guided se mantiveram elevados e instáveis, orbitando entre 1,62 e 1,72 na Máquina 2 (Tabela 6), o que a desqualifica como solução adequada para problemas onde os picos de processamento estão espremidos em concentrações locais imprevisíveis.

4.5 Discussão de Gargalos Encontrados e Limitações da Implementação

Durante a análise pormenorizada de desempenho, o principal gargalo físico identificado residiu na contenção do barramento de memória. Embora o particionamento do laço assegure a independência no cômputo lógico do algoritmo de escape-time, todas as threads necessitam gravar os seus resultados concorrentemente na matriz unificada alocada na memória RAM. Conforme o grau de paralelismo é incrementado na escalabilidade forte, essa concorrência pelo barramento de acesso à memória cria um estrangulamento que impede o sistema de atingir a linearidade perfeita no Speedup. Adicionalmente, limitações intrínsecas da topologia de hardware restringiram a aceleração teórica, notadamente o compartilhamento de Unidades Lógicas e Aritméticas (ALUs) nos núcleos que operam sob Hyper-Threading, impondo um teto de processamento simultâneo que não pode ser superado apenas via software.

Sob a ótica computacional, a própria infraestrutura do OpenMP revelou limitações incontornáveis associadas ao seu custo gerencial (overhead). Apesar de a política

dynamic ter se provado a mais eficaz na supressão do desbalanceamento de carga, a sua execução não é isenta de penalidades. O escalonamento sob demanda de blocos com granularidade mínima, como na configuração otimizada dynamic, 1, exige que o runtime do OpenMP controle continuamente a fila de requisições de trabalho, um processo de gestão que consome ciclos de CPU. Este overhead de coordenação da infraestrutura e a sincronização nas barreiras implícitas tornam-se proporcionalmente mais expressivos à medida que o tempo útil de processamento de cada thread é encurtado.

Por fim, verificou-se uma limitação nas heurísticas nativas de escalonamento do OpenMP perante a geometria fractal assimétrica do Mandelbrot. A política guided, que teoricamente ofereceria o melhor compromisso entre a minimização do overhead e a distribuição de carga, falhou ao atribuir blocos iniciais excessivamente grandes. Ao capturar precocemente as zonas mais densas de iteração, a estratégia enviesou a carga de trabalho de forma irreversível logo no arranque da rotina paralela, expondo que abordagens de decaimento exponencial de chunk não são adequadas para processamentos onde as anomalias de carga (hotspots) não seguem uma distribuição espacial linear e previsível.

4.6 Sugestões de Melhorias ou Otimizações Futuras

A despeito dos expressivos ganhos de desempenho alcançados com a paralelização via OpenMP, a análise dos gargalos evidencia oportunidades robustas para refinamentos estruturais e algorítmicos. Uma das principais frentes de otimização futura consiste na exploração do paralelismo de dados no nível de instrução, através da vetorização (SIMD - Single Instruction, Multiple Data). Considerando que a equação de recorrência do Mandelbrot não apresenta dependência de dados entre os píxeis, a aplicação de diretivas como `#pragma omp simd` no laço interno permitiria aos processadores (arquiteturas Intel de 11^a e 12^a gerações empregadas no estudo) utilizar registradores vetoriais avançados (como o AVX2 ou AVX-512). Isso possibilitaria o cômputo matemático de múltiplas coordenadas de ponto flutuante simultaneamente em um único ciclo de clock, multiplicando substancialmente o rendimento interno de cada thread.

No escopo algorítmico, a principal ineficiência da abordagem de escape-time tradicional reside no processamento intensivo de píxeis que inevitavelmente atingirão o teto máximo de iterações. Uma melhoria matemática de alto impacto seria a implementação da verificação analítica prévia do cardioide principal e do bulbo de período 2. Ao submeter as coordenadas espaciais a um simples teste geométrico antes do ingresso no laço while, o programa poderia confirmar instantaneamente a pertença do ponto ao conjunto e pular a fase de cálculo iterativo. Esta técnica cortaria pela raiz a sobrecarga das threads responsáveis pelo interior do fractal, atuando como um poderoso mecanismo de balanceamento de carga nativo e eliminando o pior cenário de processamento. Tais estratégias extrairiam o limite prático da eficiência em memória

compartilhada, elevando o teto de escalabilidade do sistema frente aos entraves físicos evidenciados nos benchmarks.

5. Conclusão

A condução da Etapa 1 deste projeto consolidou o entendimento de que algoritmos classificados como embarçosamente paralelos não garantem, por si só, uma escalabilidade linear e trivial. A geração do Conjunto de Mandelbrot evidenciou que a ausência de dependência de dados entre as iterações é apenas o requisito primário para a concorrência computacional. O verdadeiro desafio arquitetural reside na gestão da assimetria geométrica, uma vez que o custo de processamento varia de forma extrema e imprevisível ao longo do plano complexo coordenado. A utilização da interface OpenMP demonstrou ser uma ferramenta coesa e de alto nível para abstrair a complexidade da criação e sincronização de threads em sistemas de memória compartilhada, deslocando o foco analítico estritamente para a mitigação do desbalanceamento de carga.

As considerações finais sobre o desempenho atestam que a escolha da política de escalonamento determina o sucesso ou o colapso da paralelização perante cargas irregulares. A investigação empírica comprovou que o modelo de fatiamento estático, embora apresente o menor overhead de gerenciamento do sistema operacional, é fundamentalmente incompatível com o peso heterogêneo do fractal. Em contrapartida, a adoção da política dinâmica com granularidade fina (dynamic, 1) provou ser a estratégia definitiva para suprimir a ociosidade sistêmica. Ao distribuir blocos sob demanda, o escalonamento dinâmico foi capaz de absorver tanto a disparidade matemática das fronteiras do fractal quanto as diferenças físicas de operação em processadores híbridos (núcleos de performance atuando em conjunto com núcleos de eficiência), culminando em Fatores de Balanceamento perfeitamente otimizados e em ganho de velocidade (Speedup) expressivo, que alcançou até 9,5 vezes a marca da execução serial.

Em suma, a implementação atendeu aos objetivos propostos ao validar a corretude concorrente baseada no mapeamento row-major e ao expor os limites físicos modelados pela Lei de Amdahl, nomeadamente a contenção no barramento de memória na escalabilidade fraca e o teto imposto pelo Hyper-Threading. A análise rigorosa fundamentada em métricas de tempo, eficiência e proporção de carga máxima e mínima consolida o entendimento sobre a exploração plena de hardware multicore, encerrando esta etapa com uma solução computacional robusta, balanceada e profundamente otimizada para arquiteturas de memória compartilhada.

6. Referências

ORELLANA, Esbel Tomás Valero. Aula 03 - Caracterização de desempenho em programas paralelos. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. Aula 04 - Arquiteturas Paralelas. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. Aula 06 - Diretivas OpenMP. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

OPENMP SPECIFICATIONS. Disponível em: [Specifications - OpenMP](#) . Acesso em: 21 set. 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Lucas Gabriel Ferreira, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória compartilhada com OpenMP", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Google Gemini e ChatGPT. O uso se deu exclusivamente nas seguintes etapas do projeto:

- compreensão e esclarecimento de conceitos relacionados ao algoritmo de Mandelbrot, programação paralela e OpenMP;
- sugestão de possíveis melhorias e alternativas para a implementação;
- auxílio na identificação e depuração de erros encontrados durante o desenvolvimento e os testes;
- revisão e aprimoramento da redação de trechos do relatório.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 22 de setembro de 2026

Lucas Gabriel Ferreira

Matrícula: 202311126

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Laisa Pereira França, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória compartilhada com OpenMP", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Claude e ChatGPT. O uso se deu exclusivamente nas seguintes etapas do projeto:

- compreensão e esclarecimento de conceitos relacionados ao algoritmo de Mandelbrot, programação paralela e OpenMP;
- sugestão de possíveis melhorias e alternativas para a implementação;
- auxílio na identificação e depuração de erros encontrados durante o desenvolvimento e os testes;
- revisão e aprimoramento da redação de trechos do relatório.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 22 de setembro de 2026

Laisa Pereira França

Matrícula: 202311124